

FloatSOM: GPU Accelerated, Distributed, Topology-Flexible Self-Organizing Maps*

Tony Xu

John Curtin School of Medical Research, Australian National University

tony.xu@anu.edu.au

Sarah Klamt

Department of Information Technology and Electrical Engineering, ETH Zürich

Katherine Turner

Mathematical Data Science Centre, Australian National University

Anne Brüstle

John Curtin School of Medical Research, Australian National University

Felix Marsh-Wakefield†

*Centenary Institute of Cancer Medicine and Cell Biology
University of Sydney*

Givanna Putri†

Walter and Eliza Hall Institute

Reviewed on OpenReview: <https://openreview.net/forum?id=XXXX>

Abstract

GPU-accelerated Self-Organizing Map (SOM) implementations are among the most competitive options for large-scale SOM analysis, but growing dataset sizes increasingly challenge their practical use because workloads no longer fit cleanly within device-memory limits. We introduce FloatSOM, a SOM framework for scalable training and deployment that supports multi-GPU execution, out-of-memory disk-backed streaming, and scalable training-time graph topologies beyond regular lattices. We evaluate FloatSOM on 14 synthetic and real benchmark datasets together with controlled speed scaling benchmarks, and show that these graph-based topologies, combined with topology-aware hyperparameter fine-tuning, yield lower quantization error than current state-of-the-art SOM baselines. FloatSOM also sustains this performance at large scale with high-throughput distributed execution; in the largest benchmark, it trains a 1024-node SOM network on 1,000,000,000 samples with 50 features in 6.16 minutes on 8 GPUs across two separate high-performance-computing nodes.

1 Introduction

Self-Organizing Map (SOM), originally introduced by Kohonen (Kohonen, 1990), is an unsupervised machine-learning method that uses competitive learning to organize nodes such that they capture the topology of the data. In practice, this topology-preserving representation means SOMs are commonly used to map dataset topology, produce dimensionality-reduced visualizations, and conduct clustering at scale (Kangas et al., 1990). As dataset size and heterogeneity increase, the computational requirements of SOMs grow in both time and memory. Recent tools such as aweSOM have improved single-node serial-online SOM execution through CPU/GPU acceleration and ensemble stacking (Ha et al., 2025). However, many current

*LLM disclosure

†Givanna Putri and Felix Marsh-Wakefield contributed equally as co-last authors.

implementations remain constrained to single-device workloads that must fit within video random-access memory (VRAM, GPU memory), with limited support for distributed compute, out-of-core execution, and modern GPU orchestration.

SOM topology presents a second limitation. Classical SOMs are traditionally trained on regular rectangular or hexagonal lattices because fixed grids make neighborhood definition, visualization, and optimization straightforward. However, these regular lattices also impose a strong geometric prior on the learned representation. Accordingly, prior work on dynamic, growing, and graph-structured SOM variants reflects a long-standing recognition that fixed lattices are not always the best match for irregular data geometry (Alahakoon et al., 2000; Vasighi & Amini, 2017; Kangas et al., 1990). However, these alternatives have generally not been developed or evaluated in the high-throughput, large-sample regime targeted by modern GPU-enabled applications and are broadly impractical for deployment at scale.

FloatSOM is designed to address these combined systems and topology limitations. Within FloatSOM, we implement distributed multi-GPU execution, out-of-memory disk-backed streaming, and scalable topology-flexible training beyond standard fixed lattices. Specifically, we implement highly scalable minimum-spanning-tree (MST) and relative-neighborhood-graph (RNG) topologies that define the SOM neighborhood during training rather than being overlaid on a completed map. We also evaluate multiple sampling strategies as a route to further computational acceleration and derive fine-tuned hyperparameter configurations across diverse datasets and recommended operating regimes. FloatSOM is suitable for both high-performance-computing (HPC) operation and consumer-grade desktop GPUs.

2 Related Work

Related work on practical SOM deployment spans software implementations, sampling-efficient training, topology design, and model selection.

2.1 Open-Source SOM Implementations and Systems

Open-source SOM libraries range from lightweight implementations to systems-oriented packages. MiniSom implements classical serial-online training (Vettigli, 2018), while aweSOM adds CPU/GPU acceleration and ensemble stacking for large single-node workloads (Ha et al., 2025). XPySOM instead provides GPU-accelerated batch training and is the closest executable comparator to FloatSOM’s Python/GPU training pathway (Mancini et al., 2020).

Somoclu and GigaSOM provide additional parallel-systems context (Wittek et al., 2017; Kratochvíl et al., 2020). GigaSOM.jl, for example, trained a 32×32 SOM on 1,167,129,317 cells within a larger Julia analysis completed in under 25 minutes on an 11-node, 256-core CPU cluster (Kratochvíl et al., 2020). These systems differ from FloatSOM in training regime, execution model, language, or hardware, so we treat their published results as systems context rather than controlled benchmarks.

2.2 Sampling Methodologies for SOM Training

Classical online and batch SOM training schemes traditionally sample every data-point in every training iteration (Kohonen, 2001; Liu et al., 2023). Consequently, one obvious route to improving SOM speed and scalability is subsampling, which reduces the amount of data used to train the final SOM.

To date, numerous subsampling strategies have been proposed. Beyond naive random sampling, guided subsampling methods include hierarchical dynamic subset selection SOM (HDSSSOM), which concentrates computation on difficult or stale regions of the data (Wetmore et al., 2005), and adaptive sampling strategies for design-space exploration (Ito et al., 2016). However, systematically benchmarked and openly maintained implementations that compare full-data, random, and guided sampling within a modern high-performance SOM workflow remain limited.

2.3 SOM Lattice and Adaptive Topologies

Most practical SOM implementations retain regular rectangular or hexagonal lattices because they simplify neighborhood indexing, visualization, and vectorized updates (Kohonen, 1990; 2013). Hexagonal lattices are often preferred in the literature and serve as the regular-topology baseline in this manuscript (White & Kiester, 2008; Kohonen, 2013; Forest et al., 2020).

The SOM literature has also explored alternatives to fixed lattices, including dynamic maps and graph-structured neighborhoods (Vasighi & Amini, 2017; Spanakis & Weiss, 2016; Kangas et al., 1990; Jang et al., 2009). MSTs have previously appeared in SOM analyses, but prior uses generally treat the MST as an interpretive structure over an already trained map rather than as the neighborhood relation that drives SOM learning. For example, Jang et al. use MSTs for interpretation, subnode embedding, and map-shape assessment, while FlowSOM overlays an MST on trained SOM codes to visualize relationships among metaclusters (Jang et al., 2009; Van Gassen et al., 2015). This is distinct from the training-time role used here: in FloatSOM, the MST is the operative neighborhood graph during SOM updates, is recalculated from the evolving node-weight geometry, and directly changes the update influence matrix used during learning. Thus, current deployed MST uses in SOM workflows generally configure post hoc connections on trained maps, whereas FloatSOM uses MST/RNG graphs as the training neighborhood itself. A post hoc MST overlay would also leave the quantization error of the underlying trained SOM unchanged, because QE is determined by distances between samples and the fixed learned prototypes; the QE improvements reported here therefore require graph-mediated training that changes those prototype locations. Earlier SOM variants also discussed MST-defined neighborhoods during learning (Kangas et al., 1990), but these alternatives have not been assessed on large-scale datasets and do not have implementations that are either publicly available or suitable for distributed GPU computation. We have not identified prior work implementing dynamically refreshed MST or RNG topologies as scalable GPU-compatible SOM training neighborhoods. FloatSOM’s topology contribution is therefore a scalable GPU-compatible implementation and large-scale quantification of refreshed graph-based SOM training, rather than a post hoc MST overlay on a conventional trained SOM.

Relative Neighborhood Graphs (RNGs) (Toussaint, 1980) are of particular interest here. To our knowledge, dynamically refreshed RNG neighborhoods have not previously been used as a SOM training topology; we return to the full rationale and implementation for RNG in Section 3.2.2.

2.4 Hyperparameter Optimization and Fair Comparison

SOM performance depends strongly on parameters such as map size, initialization, learning-rate schedule, and neighborhood schedule. Prior work shows that these choices can substantially affect observed performance (Akinduko et al., 2016; Forest et al., 2020). Because recent SOM system comparisons often emphasize implementation-level performance against existing tools or reference configurations (Mancini et al., 2020; Kratochvíl et al., 2020), comparisons based only on untuned settings can be difficult to interpret.

More generally, modern machine-learning workflows increasingly rely on automated hyperparameter optimization rather than manual tuning alone. Tools such as Optuna provide bounded search over large hyperparameter spaces and can support multi-objective optimization, allowing parameter settings to be selected with respect to several benchmark criteria simultaneously rather than collapsed into a single score (Akiba et al., 2019). Accordingly, we test both tuned and untuned reference and FloatSOM implementations to distinguish performance under untuned settings from performance obtained under more suitable hyperparameters.

3 Methods

FloatSOM combines four methodological components: sample selection, topology definition, SOM updates, and the compute architecture used to execute them. The framework supports random, full, and HDSSOM sampling; rectangular, hexagonal, MST, and RNG topologies; and execution modes ranging from local GPU training to single-node and multi-node multi-GPU operation. Hyperparameter optimization is treated as a separate layer applied across these configurations. Figure 1 summarizes this design.

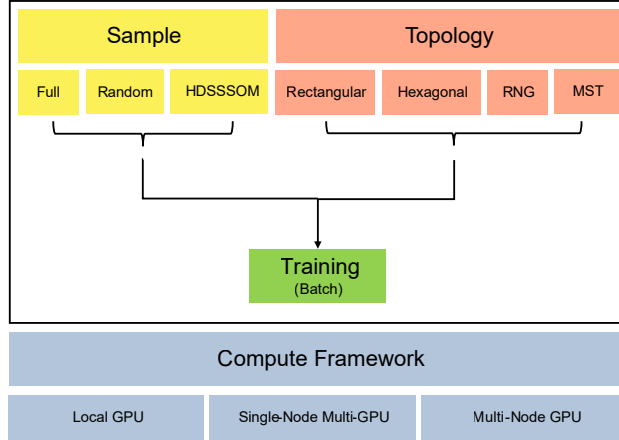


Figure 1: Schematic overview of the FloatSOM methods framing used in this manuscript. Sample selection and topology definition have configurable components that feed into the standard SOM training step, while the compute architecture determines how that same training procedure is executed in practice. The options shown here summarize the FloatSOM configurations discussed in the following subsections.

3.1 Sampling Selector Mathematics

This section formalizes the sampling policies evaluated in this manuscript.

Let the full dataset be $X = \{x_i\}_{i=1}^N$, where N is the total number of samples in the dataset. Let m denote the number of samples presented to the SOM in a given training iteration, or the sampling budget. This budget is either fixed directly or determined as a proportion ρ of the dataset:

$$m = \begin{cases} m_0, & \text{if a fixed budget is specified,} \\ \max(1, \lfloor N\rho \rfloor), & \text{if a dataset proportion is used.} \end{cases} \quad (1)$$

For index sets $\mathcal{I}_t^{(s)}$ at iteration t , the implemented full and random selectors are:

$$\mathcal{I}_t^{\text{full}} = \{1, \dots, N\}, \quad \mathcal{I}_t^{\text{random}} \sim \text{Unif}(\{I \subseteq \{1, \dots, N\} : |I| = m\}), \quad m < N. \quad (2)$$

If $m \geq N$, random returns the full dataset (no subsampling). Otherwise, to avoid biasing training toward a single fixed subsample, the random selector redraws $\mathcal{I}_t^{\text{random}}$ from the entire dataset at every training iteration. The selected training subset is $X_t^{(s)} = \{x_i : i \in \mathcal{I}_t^{(s)}\}$. Because random subsampling draws uniformly from size- m subsets, we are able to better obtain coverage over the full dataset.

HDSSSOM is implemented in multi-GPU-compatible form as an adaptive sampler that preferentially revisits difficult, under-trained, or stale samples (Wetmore et al., 2005).

3.2 Topology Definition

After initialization, neighborhood relations are defined by the selected topology. For regular-lattice baselines, we support both grid and hexagonal layouts. Consistent with prior SOM guidance, we treat hexagonal as the standard reference topology in this manuscript. We implement the hexagonal lattice topology in accordance with (Vettigli, 2018). For MST and RNG, neighborhood structure is derived from the current node-weight geometry using the methodologies below.

These alternatives change which prototypes are coupled during learning. A fixed hexagonal lattice assigns the same predetermined neighborhood pattern independently of the learned data-space geometry. Moving

one prototype can therefore influence lattice neighbors that are not locally related in data space, potentially displacing otherwise useful prototypes. An MST minimizes this coupling and gives prototypes greater freedom to redistribute along irregular or elongated structures, but its tree constraint can omit additional locally appropriate connections in dense regions. RNG is less free than MST because a node can influence several neighbors; unlike the fixed lattice, however, those additional connections are supported by the evolving prototype geometry. It can therefore remain sparse where appropriate and form a more mesh-like neighborhood in concentrated regions.

Note that regardless of the selected topology, FloatSOM uses the same node-weight initialization options. Initialization determines only the starting node weights; neighborhood relations are applied afterward according to the selected topology to establish node connections, keeping the initial state comparable across regular-lattice and graph-based runs.

3.2.1 MST Topology Implementation

The MST topology replaces fixed lattice neighborhood distance with graph hop distance on a minimum spanning tree built from current SOM nodes. The MST is therefore part of the SOM training rule: it determines which nodes receive neighborhood influence during each topology-refresh interval, rather than serving as a visualization or analysis layer after training. For P nodes in feature dimension d , the pairwise node-distance matrix is formed with the standard squared-distance Gram identity, which avoids 3D broadcast tensors and preserves $O(P^2d)$ dense linear-algebra structure.

In a standard lattice SOM, neighborhood distance is determined by fixed node grid coordinates, which are assigned before training and are independent of the learned node-weight geometry in data space. MST instead uses the node weight vectors to locate the nodes in data space, then calculates the minimum spanning tree between those node locations (Kruskal, 1956). The node distances therefore determine which tree edges are selected, while the neighborhood distance used by the Gaussian update is the graph hop distance along the resulting tree. Topology-derived influence matrices are cached and refreshed according to the topology-refresh schedule.

At the beginning of training, the topology is refreshed every iteration: the previous tree edges are discarded, pairwise distances are recalculated from the updated node weights, and the MST is recomputed from those updated node locations. The refresh rate then decays as training proceeds, so topology updates become less frequent later in training. If the current iteration does not trigger recomputation, the previous graph state and cached influences are reused. Otherwise, the MST is recalculated and the influence cache is rebuilt.

Algorithm 1 Dynamic MST topology update with refresh-triggered recomputation and cached influence reuse.

- 1: **Input:** node weights W_t , iteration t , topology-refresh policy
 - 2: **Output:** topology state (E_t, g_t , cached influences)
 - 3: Query the refresh policy for the current iteration
 - 4: **if** no topology refresh is due **then**
 - 5: return previous topology state
 - 6: **end if**
 - 7: Compute pairwise squared node distances on GPU
 - 8: Transfer distances to CPU and run Kruskal to obtain MST edges E_t
 - 9: Build adjacency from E_t
 - 10: Compute all-pairs graph hop distances g_t with chunked GPU Floyd-Warshall
 - 11: Deduplicate active radii and rebuild/update cached influence maps
 - 12: Commit E_t, g_t , and cache state
 - 13: return topology state
-

3.2.2 RNG Topology Implementation

Our RNG topology constructs a Relative Neighborhood Graph over current node distances using the standard RNG criterion (Toussaint, 1980). Unlike MST, RNG is not restricted to a single spanning-tree backbone, so multiple locally supported neighborhood relations can be retained. Conceptually, the absence of this restriction permits the RNG topology to represent both tree-like and denser mesh-like local structures simultaneously throughout the training process. Consequently, we hypothesise that this greater freedom in topology will allow RNG-based SOM maps to better represent a variety of distributions.

Algorithm 2 Dynamic RNG topology update with refresh-triggered recomputation and cached influence reuse.

- 1: **Input:** node weights W_t , iteration t , topology-refresh policy
 - 2: **Output:** topology state $(E_t, g_t, \text{cached influences})$
 - 3: Query the refresh policy for the current iteration
 - 4: **if** no topology refresh is due **then**
 - 5: return previous topology state
 - 6: **end if**
 - 7: Compute pairwise squared node distances on GPU
 - 8: Evaluate RNG candidate elimination in chunks using the blocker test
 - 9: Retain surviving RNG edges E_t and build adjacency
 - 10: Compute all-pairs graph hop distances g_t with chunked GPU Floyd-Warshall
 - 11: Deduplicate active radii and rebuild/update cached influence maps
 - 12: Commit E_t, g_t , and cache state
 - 13: return topology state
-

3.3 Multi-GPU + Larger-Than-Memory Methodology and Implementation

FloatSOM is implemented for distributed GPU execution, with support for RAM (Random Access Memory, CPU memory) spilling and disk-backed execution when datasets exceed available VRAM and RAM, respectively.

3.3.1 General Multi-GPU Logic

Distributed execution uses Ray actors with one GPU per worker and NCCL collectives for synchronous aggregation (Moritz et al., 2018). At each iteration, every worker processes its assigned shard locally in `n_chunks`, accumulates worker-local statistics across those chunks, and synchronizes once per iteration.

For worker $g \in \{1, \dots, G\}$, let $X_{t,g}^{(s)}$ denote the worker-local shard of the selected iteration subset $X_t^{(s)}$. Let j index SOM nodes, let $w_j^{(t)}$ denote the weight vector of node j at iteration t , let $b(x)$ denote the best-matching unit (BMU) of sample x under the current weights, let $h_{j,b(x)}^{(t)}$ denote the iteration- t neighborhood influence between node j and the BMU (Best Matching Unit) of x , and let η_t denote the learning rate at iteration t . The local accumulators are:

$$\begin{aligned} U_j^{(g)} &= \sum_{x \in X_{t,g}^{(s)}} \eta_t h_{j,b(x)}^{(t)} (x - w_j^{(t)}), \\ H_j^{(g)} &= \sum_{x \in X_{t,g}^{(s)}} h_{j,b(x)}^{(t)}. \end{aligned} \tag{3}$$

$U_j^{(g)}$ is worker g 's summed learning-rate-scaled displacement for node j , and $H_j^{(g)}$ is the corresponding summed neighborhood influence used to normalize that displacement.

Global synchronized accumulators are:

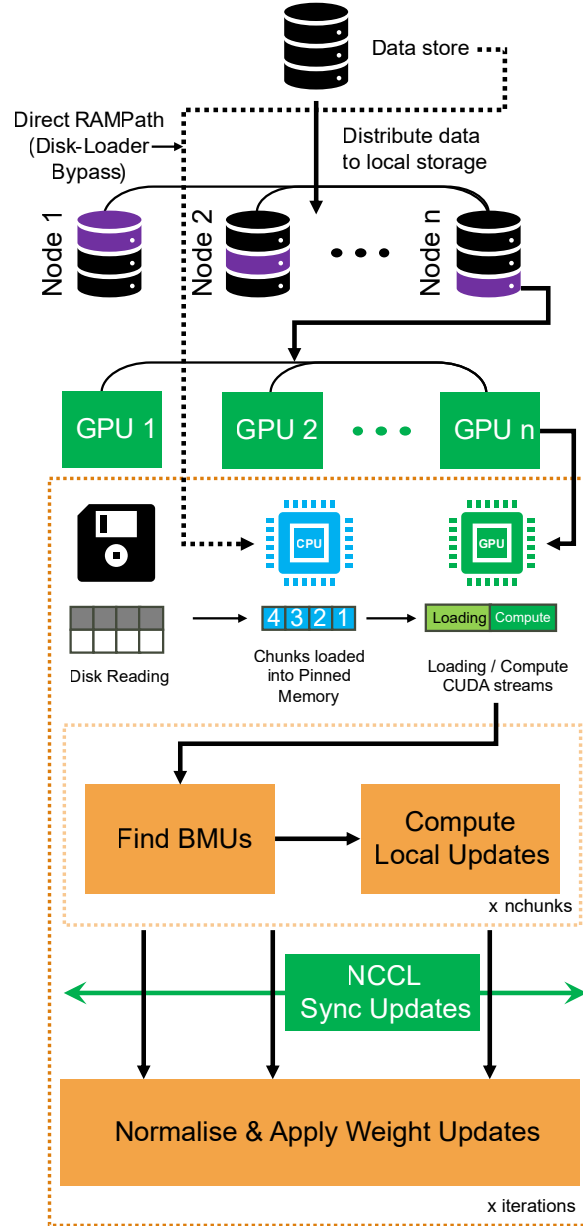


Figure 2: Multi-GPU data loading and NCCL synchronization schematic. In disk-backed mode, data are sharded to worker-local storage, loaded chunk-wise into pinned host memory, transferred to GPU with transfer overlapped with compute, processed locally, and synchronized by NCCL all-reduce before one weight update per iteration. In RAM mode, data are instead pre-sharded into worker-local CPU RAM and follow the same pinned-memory-to-GPU path without disk reads.

$$\begin{aligned}
U_j &= \sum_{g=1}^G U_j^{(g)}, \\
H_j &= \sum_{g=1}^G H_j^{(g)}.
\end{aligned} \tag{4}$$

In implementation, $U_j^{(g)}$ and $H_j^{(g)}$ are accumulated across the worker’s `n_chunks` and synchronized once per iteration. After synchronization, U_j and H_j define the global update numerator and normalization denominator for node j . Under weighted normalization, the normalized update is given by U_j/H_j up to numerical safeguards; optional momentum is then added to this normalized update before applying the resulting change to the node weight $w_j^{(t)}$.

For P nodes and d features in float32, the two update accumulators contain $Pd + P$ values, so their logical all-reduce payload is $4P(d + 1)$ bytes per worker per iteration, plus one 4-byte sample count. A ring all-reduce sends and receives approximately $2(G - 1)/G$ times that payload per worker for G workers. This communication is independent of the number of observations N ; increasing N instead increases worker-local computation and data staging. With observation chunks of at most C rows, the dominant bounded worker arrays scale as $O(Cd + Pd)$ rather than requiring the full $O(Nd)$ dataset in GPU memory. Graph-topology distance and influence structures add terms that scale with P^2 and can be tiled or spilled when necessary, which explains why increasing node count is a less favorable scaling regime than increasing sample count.

3.3.2 Multi-GPU Implementation Details

As shown in Fig. 2, each worker uses a chunked loading path from CPU memory to GPU memory. In streaming mode, data are distributed to worker-local disk shards and then read chunk-by-chunk into pinned host memory before transfer to GPU; in RAM mode, data are pre-sharded into worker-local CPU RAM and follow the same pinned-memory path without disk reads. FloatSOM overlaps data transfer and compute across CUDA streams, uses JIT-compiled kernels (Just-In-Time) for the core BMU and update steps, and synchronizes worker-local accumulators once per iteration with NCCL all-reduce. Weights therefore remain resident on worker GPUs across iterations. For large topologies that would otherwise exceed VRAM, FloatSOM also supports spilling the associated graph-distance or influence structures from VRAM to system RAM.

3.3.3 Larger-Than-Memory Capable Topology Updates

Additional larger-than-memory support for topology updates is provided through topological chunking. Topological chunking applies the same idea to topology-side computations within each worker. When graph-distance or influence structures would otherwise exceed a worker’s VRAM, those computations are tiled and evaluated in bounded pieces rather than materialized at once. This topology-side chunking is not depicted in the Fig. 2 data path, but it follows the same per-worker bounded-memory execution rule.

3.4 Multi-Objective Hyperparameter Optimization

We use Optuna as an automated multi-objective hyperparameter optimization tool to derive near-optimal performance and corresponding hyperparameters for each FloatSOM configuration under a given sampling and topology combination (Akiba et al., 2019).

4 Experimental Setup

We use two benchmark protocols: an Optuna quality benchmark and a speed scaling benchmark. The first evaluates algorithmic quality and tuned attainable performance, and the second evaluates runtime and distributed scaling behavior. All production Optuna and speed benchmarks reported in this manuscript

were executed on Gadi at the National Computational Infrastructure (NCI), Australia, on gpuvolta nodes (National Computational Infrastructure, n.d.), using a consistent multi-GPU environment across runs.

4.1 Optuna benchmark protocol

We use Optuna-based multi-objective optimization to determine the best attainable performance and corresponding hyperparameters for each sampling (full and random) and topology (hexagonal, MST, and RNG) combination, optimizing jointly for train and holdout quantization error (QE_T and QE_H). In this protocol, each dataset-topology-sampling configuration is optimized for 200 trials across 10 seeds under variant-appropriate search constraints. The reported runs used the standard in-memory batch path rather than the Ray-distributed execution stack. This corresponds to:

$$14_{\text{Datasets}} \times 10_{\text{Seeds}} \times 3_{\text{Topologies}} \times 2_{\text{SamplingMethods}} \times 200_{\text{Trials}} = 168,000 \text{ Runs}$$

The neighborhood-radius search space was shared across topology families. In particular, `initial_radius` was an Optuna-optimized parameter for hexagonal, MST, and RNG runs, with the same search interval of 0.5 to 10.0 in each case. Thus, the hexagonal topology comparisons below use a tuned hexagonal comparator rather than a default-radius hexagonal baseline.

The main sampling comparison in Section 5.2 compares full versus random sampling within matched hexagonal runs. Topology comparisons in Sections 5.3-5.4 use full-sampling runs across hexagonal, MST, and RNG. Otherwise, the remaining quality analyses use full sampling. A separate focused HDSSOM pilot is reported in Section 5.2 and summarized in Supplementary Table S2. This pilot additionally included the HDSSOM sampling methodology alongside the full and random sampling methodologies.

4.1.1 Optuna benchmark datasets and preprocessing

The Optuna benchmark uses 14 synthetic and real scikit-learn datasets spanning low- and high-dimensional regimes (Pedregosa et al., 2011); dataset metadata are listed in Supplementary Table S1. Synthetic datasets are generated according to the random seed, whereas real datasets are loaded directly from sklearn. Across the Optuna protocol, inputs are standardized feature-wise to zero mean and unit variance using `StandardScaler`, then deterministically permuted and split into fixed 70/30 train-holdout partitions. We retain both partitions to evaluate both training-set representation and transfer to unseen samples.

4.1.2 Optuna quality metrics

Our primary quality metric is Quantization Error (QE) (Kohonen, 2001). We report QE on both the training partition (QE_T) and the holdout partition (QE_H). The training value reflects how well the fitted SOM represents the samples it was trained on, whereas the holdout value evaluates the same trained map after projecting previously unseen samples onto it. The holdout partition is therefore not used during fitting; it is used only to evaluate the trained map, analogous to a test set in conventional model-training workflows.

Balanced QE , denoted QE_B , is defined as the mean of QE_T and QE_H . QE_B is therefore a composite endpoint that weights data representation fidelity (train) and generalizability (holdout) equally. Note that in the Optuna runs, QE_T and QE_H are optimized jointly as a two-objective vector, with QE_B only calculated *post hoc*.

We quantify local BMU-neighborhood ordering using Mean Tied Rank (MTR) (Ramos et al., 2018). For each sample x_i , non-winning units are ranked by graph distance from the first BMU, $b_i^{(1)}$, with tied distance shells assigned their average ordinal rank. If the second BMU, $b_i^{(2)}$, lies in shell S_d and L_d units lie in closer shells, its tied rank is $\tau_i = L_d + (|S_d| + 1)/2$, and $MTR = N^{-1} \sum_i \tau_i$. Lower MTR indicates closer local ordering between the first and second BMUs.

MTR is used instead of topographic error because one-hop adjacency is not equivalent across hexagonal, MST, and RNG graphs; topographic error also depends on lattice properties and map design (Ramos et al., 2018; Neme & Miramontes, 2005; Machón-González & López-García, 2010). We additionally report node

utilization, defined as the fraction of nodes selected as a BMU, and its complement, dead-node fraction. These post hoc diagnostics are computed on training and holdout splits and balanced as for QE .

4.1.3 XPySOM calibration protocol

We calibrated the FloatSOM batch pathway against XPySOM using the 14 benchmark datasets, 10 shared seeds, full sampling, a 32×32 map, and one run per dataset–seed condition without top- k selection. Both implementations used the same deterministic 70/30 train–holdout split, number of batch epochs, and XPySOM-like untuned hyperparameters. XPySOM’s initialized node weights were transferred into FloatSOM after reconciling the two implementations’ node-coordinate ordering, so paired runs began from corresponding prototypes. The primary implementation-concordance comparison used the hexagonal topology in both implementations; the associated MST and RNG FloatSOM paths retained the same XPySOM hexagonal reference to quantify the additional effect of changing the FloatSOM training topology.

We recorded train, holdout, and balanced QE together with training time. Within each dataset, paired percentage effects across the 10 seeds were summarized using the Wilcoxon signed-rank location estimate, its exact distribution-free 95% interval, and a two-sided paired Wilcoxon test; global rows pool the 140 dataset–seed pairs. Supplementary Figs. S1–S3 and Supplementary Tables S4–S6 report these results. These are difference tests rather than formal equivalence tests: a non-significant hexagonal comparison is interpreted as no implementation-associated QE difference detected, because no equivalence margin was prespecified.

4.2 Speed scaling benchmark protocol

The speed scaling benchmark evaluates runtime and distributed scaling behavior in different compute and algorithm configurations. Speed scaling is evaluated with runs across $G \in \{1, 2, 4, 8\}$ GPUs under a series of fixed scaling protocols. Runtime summaries are computed from repeated executions per configuration and reported as both absolute training time and efficiency relative to the corresponding 1-GPU comparison. We compute scaling efficiency as $E_G = (T_1/T_G)/G \times 100\%$, where T_1 is the 1-GPU runtime and T_G is the runtime on G GPUs. A value of 100% corresponds to ideal linear scaling; lower values indicate that communication, orchestration, or I/O overhead has reduced the realized speedup, while values above 100% can occur when additional GPUs also change the memory or data-staging regime. These scaling runs use the Ray-orchestrated distributed execution layer built on top of the standard FloatSOM training path (Moritz et al., 2018). Accordingly, the scaling figures in Sections 6.1-6.2 and the runtime/scaling comparison reported later against XPySOM should be interpreted as distributed-execution results rather than the in-memory Optuna path.

For scaling-efficiency calculations, when a single-GPU baseline was missing at a given axis value due to timeout, we estimated that baseline by local linear extrapolation from the last available 1-GPU point on the same curve. That is, runtime was assumed to scale proportionally with the axis variable for the extrapolation step (for example, doubling sample count or doubling dimensionality doubles the estimated 1-GPU runtime).

Topology speed comparisons include hexagonal, MST, and RNG, with harmonized workload settings so ratios isolate topology-associated runtime effects. A dedicated random versus full comparison, run on $G \in \{1, 2, 4\}$ GPUs, is additionally included to isolate sampling-specific runtime effects independently of the multi-GPU scaling runs.

XPySOM was selected as the executable external baseline because it most closely matches FloatSOM’s Python/GPU batch-training regime. We also attempted aweSOM on the standard speed workload, but all five runs reached the 1800-s timeout before completing 60% of N online updates. Matching FloatSOM’s 10 batch iterations would require $10N$ pointwise updates, so aweSOM was not included in the timed comparison.

4.2.1 Scaling benchmark datasets

The speed benchmark uses synthetic random matrices with uniform values in $[0, 1]$. No train-holdout split is used here because the objective is runtime rather than generalization, so all generated data are used for training. We use a shared default configuration of 10^7 samples, 50 dimensions, a 32×32 SOM grid, and 10

training iterations, and then vary one workload axis at a time: sample count (10^6 – 10^9), feature dimension (50–5000), or grid side length (8–64). The exact axis values used for each scaling panel are provided in Supplementary Table S3.

Additional CPU and RAM resources attached to each GPU are scaled linearly with GPU count while keeping the software environment and benchmark procedure consistent across runs. For scaling benchmarks, each run is timed out if it exceeds 30 minutes of wall time. Per-GPU resource allocation is fixed at one NVIDIA V100 (32 GB VRAM), 12 CPU cores, 90 GB system RAM, with one full node comprising 4 GPUs and their associated CPUs, and with 400 GB of associated local disk storage.

4.3 Hyperparameter Tuning and Stability

To quantify tuning benefit, we extracted the best Optuna settings for each sampling-topology combination and formed deployable configurations from the mean numeric and modal categorical values. These configurations were derived from winning groups among datasets with at least 1000 samples, matching the sample-size regime expected for most applications.

The stability analysis focuses on four tuned hyperparameters that govern SOM training dynamics. These settings determine how the map is initialized and how updates evolve over training: the initial radius sets the early neighborhood scale around each BMU, the initialization method sets the starting node weights, the radius decay type controls the shift from broad global organization toward local refinement, and the momentum-use parameter determines whether each update retains part of the previous update direction.

4.3.1 Tuned Configuration versus Untuned Reference Analysis

Fixed configurations were rerun under matched dataset, seed, topology, and split keys. The final topology diagnostic comprised all 14 benchmark datasets, 20 shared random seeds, full sampling, and each of the hexagonal, MST, and RNG topologies, giving 280 dataset–seed units per paired contrast. Preprocessing and the deterministic 70/30 train–holdout split followed Section 4.1.1. The untuned profile used the same XPySOM-like settings for all topologies: initial radius 5, random initialization, exponential radius decay, initial momentum 0.5, and momentum disabled. The tuned profile used the fixed topology-specific full-sampling configurations derived above: initial radii of 1.0266, 1.4639, and 1.4148 and initial momenta of 0.6069, 0.6146, and 0.6127 for hexagonal, MST, and RNG, respectively, with random initialization, asymptotic radius decay, and momentum enabled. Deployment analyses compare these tuned FloatSOM configurations with untuned hexagonal XPySOM, while topology diagnostics compare tuned and untuned FloatSOM within and between topologies. These reruns estimate deployable fixed-configuration performance; the top- k Optuna summaries instead estimate attainable performance within the search budget.

For Supplementary Fig. S5, tuned and untuned values were paired within dataset, seed, topology, and split. Supplementary Table S12 pools the 280 matched dataset–seed differences for each prespecified profile–topology–metric contrast and reports the mean paired effect, two-sided one-sample paired t -test confidence interval, Cohen’s d_z , raw p-value, and win/loss/tie counts. These raw p-values are descriptive diagnostics rather than a multiplicity-adjusted family; effect sizes and confidence intervals are the primary summaries. Supplementary Table S13 instead reports the corresponding mean observed metric within each profile, dataset, and topology across the 20 seeds. Dataset-level forest-plot results use the convention described above, and overall rows pool all matched pairs with the same paired t -test and confidence-interval calculation.

4.3.2 Hyperparameter stability and dataset-type stratification

Hyperparameter stability is important because it determines whether a method can be deployed reliably without bespoke retuning. We therefore extracted the top-ranked tuned Optuna trial separately for each seed and compared the recovered hyperparameter values across seeds. We define hyperparameter stability as the variation in these recovered settings. For numeric parameters, stability is measured by the relative difference between runs, $|a - b| / \max(|a|, |b|, \varepsilon)$, where a and b are the values of a given parameter for the two compared seeds and $\varepsilon = 10^{-12}$. These relative differences are then averaged across numeric parameters,

with lower values indicating higher stability. For categorical parameters, stability is defined as the mismatch rate across the same seed pairs. We report both mean per-parameter stability and an equal-weight overall stability summary across datasets.

4.3.3 Matched initial-radius sensitivity analysis

To determine whether topology differences could be explained by topology-specific treatment of neighborhood radius, we performed a controlled initial-radius sweep. Hexagonal, MST, and RNG maps were evaluated at $r \in \{0.5, 0.75, 1.0266, 1.5, 2, 3, 5\}$ using the same 20 random seeds on each of the 14 benchmark datasets. The value $r = 1.0266$ was the selected hexagonal full-sampling radius and is denoted **Hex optimal** in Fig. 9. Apart from topology and the deliberately varied initial radius, the training configuration was fixed: full sampling, random initialization, asymptotic radius decay, momentum enabled with initial momentum 0.6069, and XPySOM-compatible normalization. Runs were therefore matched by dataset and seed, ensuring that topology families received the same radius values rather than different search ranges or topology-specific radius schedules.

We evaluated balanced QE , balanced MTR, and balanced node utilization. Because absolute QE scales differ substantially among datasets, each dataset–seed QE_B value was divided by the matched hexagonal QE_B at $r = 1.0266$. Ratios were averaged on the log scale across seeds within each dataset and then across datasets with equal dataset weight; exponentiating this mean yields the geometric-mean ratio shown in Fig. 9A. Balanced MTR and balanced node utilization are retained in their observed units in Fig. 9B and Fig. 9C, respectively. Along each topology’s response curve, direction-aware percentage changes from $r = 1.0266$ were calculated for all 280 matched dataset–seed units and tested against zero using two-sided one-sample t -tests; Benjamini–Hochberg adjustment was applied across the 54 non-anchor radius–topology–metric tests. These tests assess within-topology radius response.

We additionally conducted post hoc cross-topology QE_B contrasts at each radius. For every topology pair, dataset, seed, and radius, we calculated the log ratio of the comparator QE_B to the reference QE_B , averaged log ratios across the 20 seeds within each dataset, and applied a two-sided one-sample t -test to the 14 equally weighted dataset effects. Exponentiated means and confidence limits are geometric-mean QE_B ratios, with values below 1 favouring the comparator. Benjamini–Hochberg correction was applied across all 21 topology–pair–radius contrasts. Supplementary Table S15 reports the estimates, intervals, raw p-values, and adjusted q-values.

4.3.4 Execution-path concordance analysis

To determine whether the quality diagnostics depended on execution pathway, we compared local CuPy and Ray streaming under matched inputs and fixed tuned configurations. The complete protocol is provided in Supplementary Methods S1, and the paired results are reported in Supplementary Table S14. Because no equivalence margin was prespecified, non-significant results are interpreted as no difference detected rather than proof of formal equivalence.

4.4 Statistical analysis

All Optuna comparisons use matched pairs within dataset, seed, and split units to control for substantial between-run heterogeneity. Within each matched unit, trials were ranked by the target metric, the top five were retained, and each condition was summarized by the median of those retained trials, yielding a top- k summary with $k = 5$ intended to estimate near-optimal attainable performance under a fixed number of tuning trials, where each trial is one candidate hyperparameter configuration evaluated by Optuna. Paired effects were then computed as simple condition differences, with negative values favoring the first condition for lower-is-better metrics. Dataset-level and global summaries are shown as forest plots with 95% confidence intervals from two-sided paired one-sample t -tests. Supplementary Fig. S4 repeats the topology comparisons at $k \in \{1, 3, 5, 10\}$, always using the median of the retained trials within each matched unit, to assess sensitivity to the number of near-optimal trials retained.

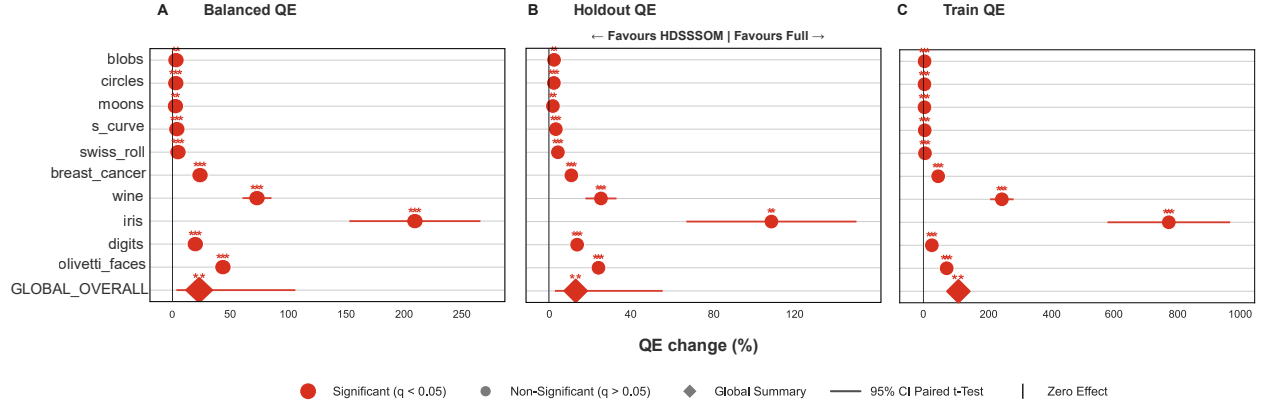
Figure 3: Full versus HDSSOM on Pilot Dataset

Figure 3: HDSSOM screening pilot on QE_B (hexagonal topology): full vs HDSSOM, using the smaller pilot configuration summarized in Supplementary Table S2 (10 datasets, 5 seeds, with all other pilot settings held fixed). Panels report dataset-matched paired top- k within-unit medians plus dataset-level paired-effect summaries (Section 4.4). Forest whiskers denote 95% paired t -test confidence intervals around the mean paired effect.

We report Benjamini-Hochberg q -values alongside raw p -values for related dataset-level tests, and use q -values for figure markers and significance counts. Each topology contrast in Figs. 6-7 forms a separate family of 42 tests (14 datasets across QE_B , QE_H , and QE_T). Pooled overall tests are reported separately with raw p -values.

5 Results

5.1 XPySOM calibration

Under matched-configuration XPySOM versus FloatSOM calibration on hexagonal QE (Fig. S3), no implementation-associated QE difference was detected. Accordingly, we use hexagonal FloatSOM batch as the operational XPySOM reference in the benchmarks that follow, while not claiming formal statistical equivalence. The runtime differences are attributable to FloatSOM’s JIT kernels, which incur a small startup cost. This overhead is progressively amortized as workload size increases, after which FloatSOM runs faster than XPySOM on larger datasets.

5.2 Comparison of Different Sampling Methods

We first report a focused HDSSOM pilot as an elimination comparison rather than as part of the broader sampling benchmark. This pilot used a smaller, more limited configuration than the later full versus random analysis and is summarized in Supplementary Table S2.

Under that pilot configuration, HDSSOM was materially worse than full sampling. In the hexagonal view shown in Fig. 3, full outperformed HDSSOM in all 50 paired comparisons (10 datasets $\times$ 5 seeds; 50 wins, 0 losses, 0 ties), with dataset-level median balanced- QE improvements ranging from 2.5% to 209.7% and a global median improvement of 38.7%. These HDSSOM results should therefore be interpreted as a pilot screen under the stated configuration rather than as a comprehensive evaluation of all possible HDSSOM schedules. Given these large differences, we focused on the full and random sampling methodologies for the remainder of the manuscript.

The full versus random analysis in Fig. 4 was generated from matched hexagonal Optuna runs in which the sampling selector was switched from full to random. Effects were computed within matched seed-specific units and then summarized across seeds. Fig. 4A-C summarize the matched full versus random paired effects

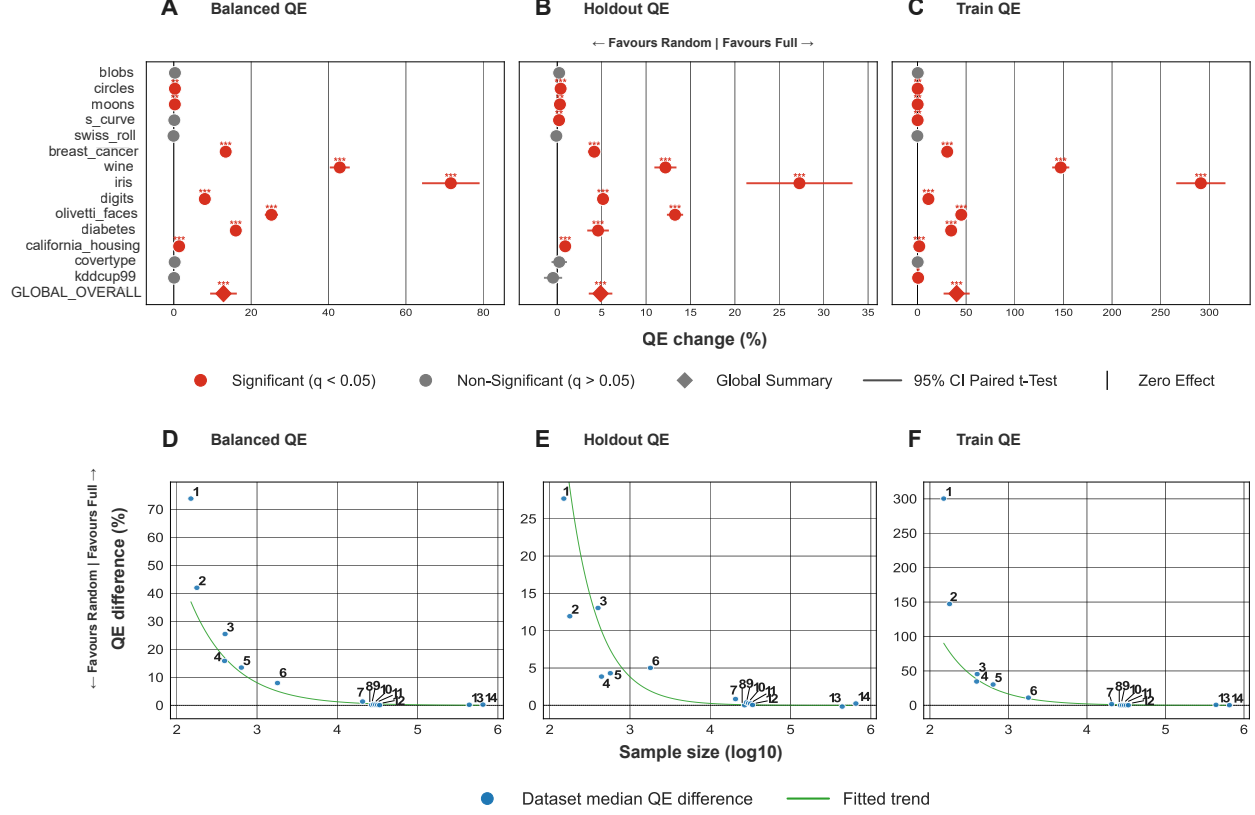
Figure 4: Full vs Random Outcomes and Dataset-Size Regression (Hexagonal)

Figure 4: Full versus random sampling under the hexagonal topology. Panels A-C report paired full-versus-random effects for QE_B , QE_H , and QE_T . Panels D-F show the corresponding dataset-size regressions for the same three metrics, with numbered datasets keyed in Supplementary Table S1. Figure significance markers use Benjamini-Hochberg adjusted q-values. In the larger-dataset regime observed here ($>10,000$ samples), little paired QE separation is detected; at smaller dataset scales, random is more variable and less stable.

for balanced, holdout, and train QE , respectively, showing that full sampling generally provides equal or better QE than random sampling. Notably, this full versus random separation is much smaller than the full versus HDSSOM pilot effect shown in Fig. 3; in most dataset-level comparisons, the full versus HDSSOM improvement is at least twice as large - as evidenced by the difference in axis scales between Fig. 3 and Fig. 4.

We find the effectiveness of random sampling relative to full sampling is scale-dependent. Above 10,000 samples, paired QE differences are not meaningfully detected, whereas in smaller datasets the random arm shows higher variability and less stable outcomes (Fig. 4D-F). This is consistent with reduced per-iteration sample support under random subsampling (Fig. 4D-F). In this benchmark, the $> 10,000$ regime is therefore a useful practical proxy for more stable random sampling behavior.

Nevertheless, full sampling remains the best sampling strategy for optimal QE results. Accordingly, all remaining analyses reported below rely on full sampling unless specified.

5.3 Topology Results

Topology comparisons use QE as the optimized endpoint and hexagonal batch SOM as the regular-topology baseline; Fig. 5 illustrates the three neighborhood structures. `initial_radius` was optimized over the same interval for every topology. The selected full-sampling values were 1.03 for hexagonal, 1.46 for MST, and

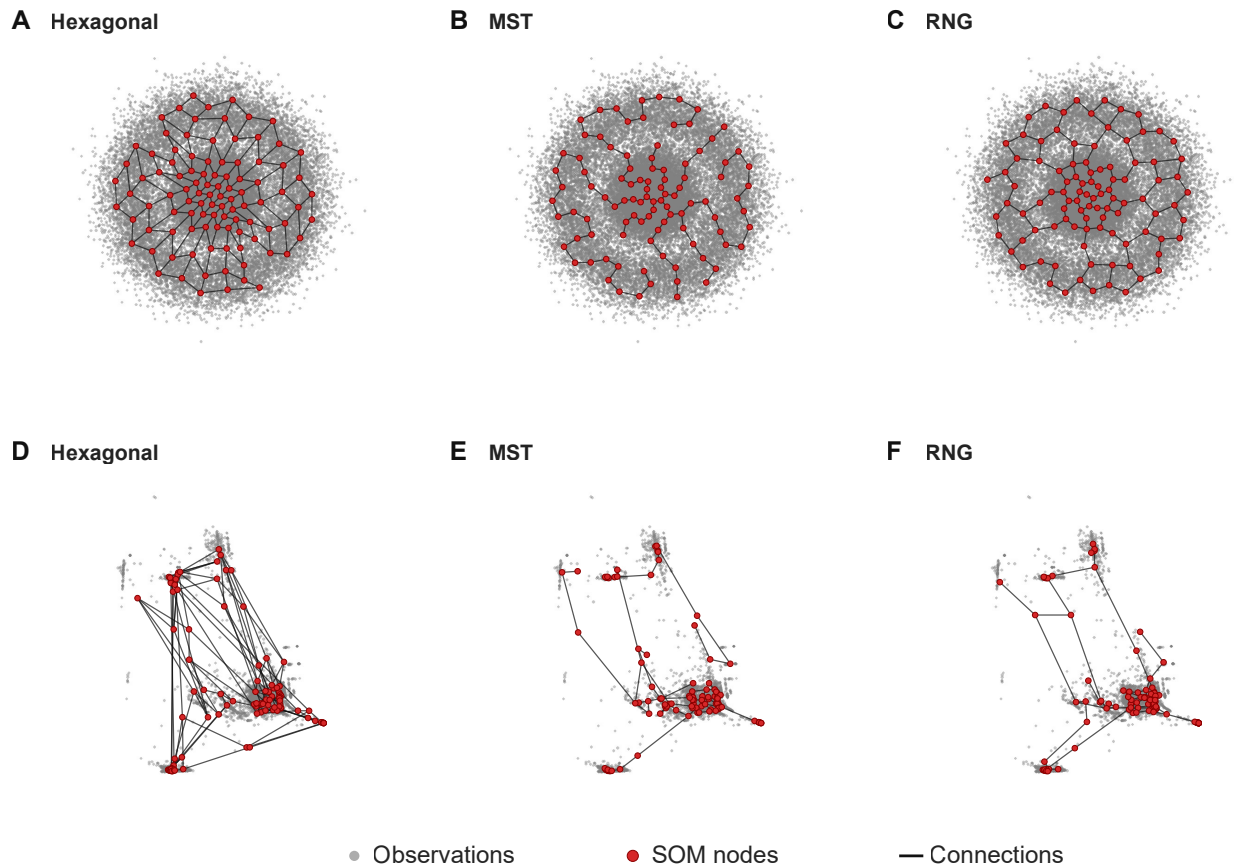
Figure 5: Representative Topologies

Figure 5: Representative node and connection overlays for matched 100-node hexagonal, MST, and RNG SOMs. A–C: circles in native 2D. D–F: KDD Cup 99, trained on standardized 41-dimensional observations and displayed using one shared 2D PCA projection fitted to the observations and applied to all three sets of prototypes. Training used seed 42, full sampling, random initialization, 50 iterations, and the untuned XPySOM-like settings used at this point in the manuscript: initial learning rate 0.5, initial radius 5 with exponential decay, momentum disabled, and XPySOM-compatible normalization. Grey observation clouds show a deterministic maximum of 30,000 points; black lines are topology connections and red markers are SOM nodes. PCA can distort graph geometry in the original 41-dimensional space.

1.41 for RNG; under random sampling they were 1.17, 1.82, and 1.77. Thus, the graph-topology gains are not comparisons against a broader default-radius hexagonal map, although radius was not isolated from the other tuned parameters.

In the KDD Cup 99 projection (Fig. 5D–F), the hexagonal topology contains several apparently nonlocal connections spanning separated regions of the projected prototype distribution, whereas the MST and RNG connections more closely follow its local geometry. This pattern illustrates how predetermined lattice neighbors can couple the updates of prototypes that are not locally adjacent in the displayed data structure, restricting how independently the nodes can redistribute. Because the display is a two-dimensional PCA projection of the 41-dimensional training space, this visual pattern is illustrative rather than direct evidence of nonlocality in the original space.

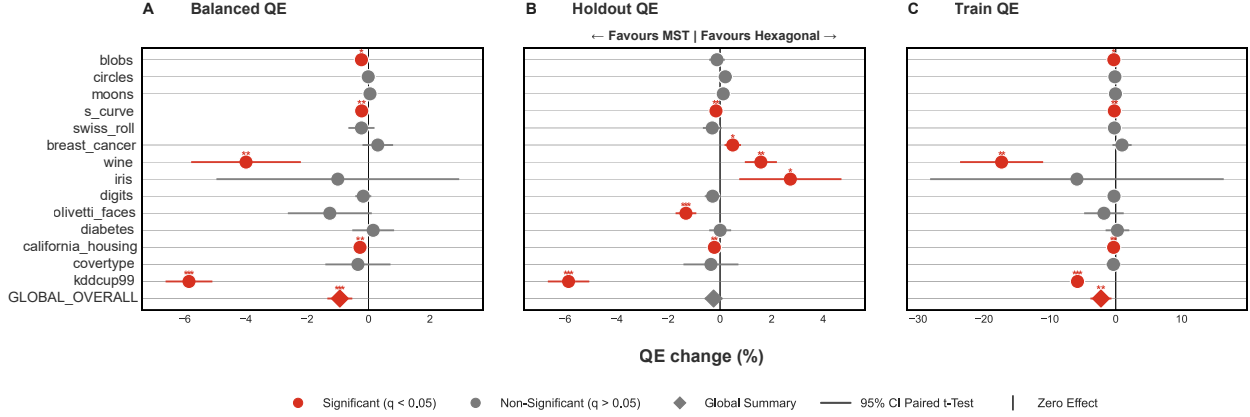
Figure 6: Hexagonal vs MST Across QE Metrics

Figure 6: Hexagonal versus MST topology on QE metrics under full sampling only. Panels A-C report paired full sampling only QE effects for QE_B , QE_H , and QE_T across the available full sampling datasets. Forest whiskers denote 95% paired t -test confidence intervals around the mean paired effect. Supplementary Table S7 reports the per-dataset effect estimates, 95% confidence intervals, raw p-values, and dataset-level Benjamini-Hochberg adjusted q-values.

5.3.1 MST

We report dataset-wise paired improvement summaries (hexagonal over MST) using the statistical procedure in Section 4.4 in Fig. 6.

Overall, MST outperforms matched hexagonal on Balanced QE (Fig. 6A), indicating a net advantage across train and holdout performance. This aggregate gain is driven more clearly by Train QE (Fig. 6C), while Holdout QE is more mixed across datasets (Fig. 6B) and shows no clear overall holdout advantage. The overall paired t-test p-values are Balanced QE ($p=1.12e-05$); Holdout QE ($p=0.15$); and Train QE ($p=0.0064$). Supplementary Table S7 lists the corresponding per-dataset and overall hexagonal-comparison effect estimates, 95% confidence intervals, raw p-values, and Benjamini-Hochberg q-values for MST and RNG.

5.3.2 RNG

To evaluate RNG topology performance, we reuse the paired reporting logic on hexagonal versus RNG, again centered on QE_B with QE_H and QE_T in Fig. 7.

RNG has lower QE than matched hexagonal on the reported QE endpoints (Fig. 7A-7C), with overall paired t-test p-values of Balanced QE ($p=7.4e-10$); Holdout QE ($p=0.0232$); and Train QE ($p=4.69e-06$). Supplementary Table S7 lists the corresponding per-dataset and overall hexagonal-comparison effect estimates, 95% confidence intervals, raw p-values, and Benjamini-Hochberg q-values for MST and RNG.

The main trend in Fig. 7 is that RNG improves on hexagonal most clearly in balanced QE and especially in train QE, with the separation most apparent in the real and larger datasets where the added flexibility of the graph neighborhood appears more useful than the fixed regular lattice.

5.3.3 Direct MST–RNG comparison

Fig. 8 directly compares MST and RNG under full sampling. The two graph topologies are close on QE , and neither dominates across all three endpoints.

Figure 7: Hexagonal vs RNG Across QE Metrics

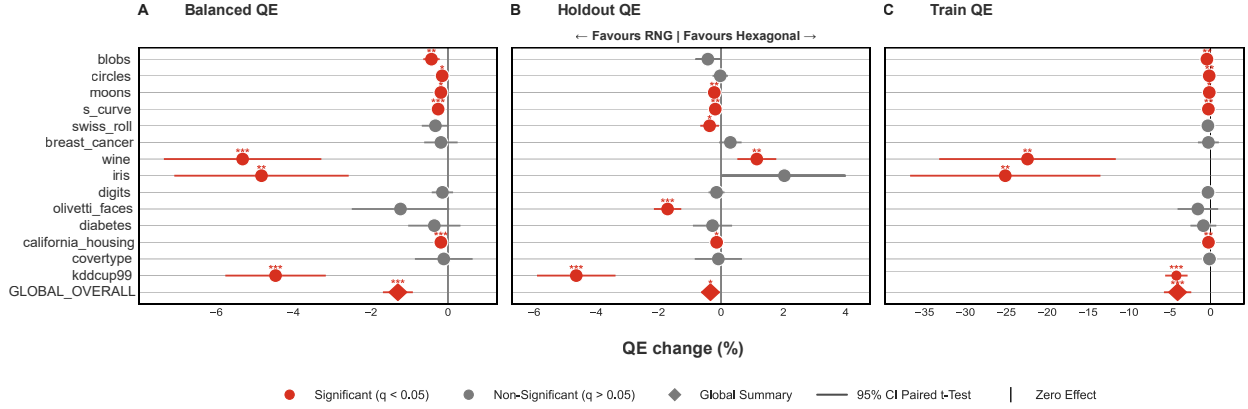


Figure 7: Hexagonal versus RNG topology on QE metrics under full sampling only. Panels A-C report paired full sampling only QE effects for QE_B , QE_H , and QE_T across the available full sampling datasets. Forest whiskers denote 95% paired t -test confidence intervals around the mean paired effect. Supplementary Table S7 reports the per-dataset effect estimates, 95% confidence intervals, raw p-values, and dataset-level Benjamini-Hochberg adjusted q-values.

Figure 8: MST vs RNG Across QE Metrics

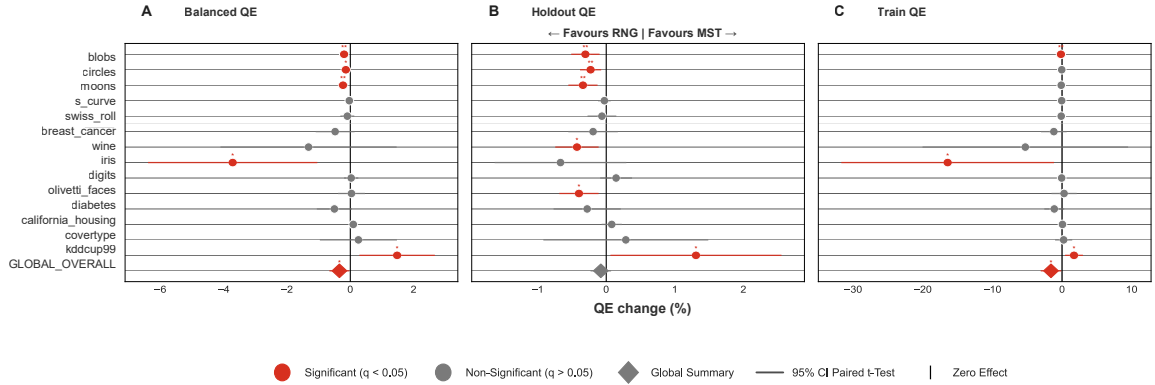


Figure 8: MST versus RNG topology on QE metrics under full sampling only. Panels A-C report paired full sampling only QE effects for QE_B , QE_H , and QE_T across the available full sampling datasets. Forest whiskers denote 95% paired t -test confidence intervals around the mean paired effect.

5.3.4 Initial-radius sensitivity

Because initial radius was selected during tuning and differed across topology families, we isolated its effect in a fixed-configuration sweep. All non-radius hyperparameters were held at the tuned hexagonal full-sampling configuration, and each of seven radii was evaluated for hexagonal, MST, and RNG maps using the same 20 seeds on all 14 datasets. Fig. 9A reports QE_B after normalizing every dataset-seed condition to its matched hexagonal value at the selected radius ($r = 1.027$). We summarize these ratios by averaging log ratios across seeds within each dataset, weighting the 14 datasets equally, and exponentiating the cross-dataset mean. The resulting geometric-mean ratio is dimensionless: 1 denotes matched hexagonal performance at the selected radius, values below 1 indicate lower QE_B , and values above 1 indicate higher QE_B .

The observed QE_B minimum occurs at $r = 0.75$ for hexagonal and at the larger radius $r = 1.5$ for both MST and RNG (Fig. 9A). For MST and RNG, however, the response is flat between $r = 1.027$ and $r = 1.5$:

the within-topology differences from the anchor are not significant after adjustment (MST $q=0.35$; RNG $q=0.21$). Thus, $r = 1.5$ is the lowest observed point in this grid rather than evidence of a sharply identified optimum. The post hoc matched topology contrasts support the separation in the useful radius region: at $r = 1.027$, MST and RNG have 1.51% and 1.85% lower geometric-mean QE_B than hexagonal ($q=0.036$ and $q=0.035$), increasing to 3.30% and 3.67% at $r = 1.5$ ($q=0.016$ and $q=0.014$; Supplementary Table S15). No RNG–MST contrast is significant at any tested radius. The graph-topology advantage over hexagonal therefore persists under identical radius treatment, while the two graph topologies remain close competitors on QE_B .

The sweep also reveals a pronounced topology-specific quantization–ordering trade-off. In Fig. 9B, increasing radius sharply lowers hexagonal MTR but simultaneously and progressively worsens its QE_B in Fig. 9A. Around the useful $r = 1.027$ – 1.5 region, MST and RNG do not show the same penalty: their QE_B remains essentially flat, while RNG MTR improves and MST MTR remains stable. At still larger radii, MST and RNG eventually also exchange higher QE_B for lower MTR, but this trade-off is delayed and substantially weaker than for hexagonal maps. The adaptive graph topologies therefore preserve a more favourable joint QE /MTR operating region rather than merely shifting the same hexagonal response to a different radius.

Fig. 9C provides the complementary map-capacity check. Hexagonal node utilization falls from 0.874 at the selected radius to 0.859 at $r = 1.5$ and approximately 0.846 at $r \geq 2$. By contrast, MST and RNG remain near 0.89 throughout the $r = 1.027$ – 2 region in which their QE_B response is also comparatively stable. These utilization results do not support poorer use of the available SOM nodes as an explanation for the graph topologies’ lower QE_B .

RNG also has lower observed MTR than MST throughout the sweep (Fig. 9B); for example, at the hexagonal selected radius, mean balanced MTR is 5.10 for RNG and 8.06 for MST. This agrees with the matched topology diagnostics, in which RNG has the stronger MTR result. The significance symbols in Fig. 9 should not be read as MST–RNG comparisons: they compare each non-anchor radius with $r = 1.027$ within the same topology. The panel therefore separates two findings: RNG maintains better local ordering than MST in absolute MTR, while changing radius away from the anchor produces topology-specific within-curve changes.

The fixed-configuration diagnostics provide the corresponding MTR and node-use comparison (Table 1). MST and RNG both improved balanced QE relative to hexagonal maps with and without tuning, but only RNG clearly lowered untuned balanced MTR. Under tuning, no balanced- QE difference was detected between MST and RNG, while RNG lowered balanced MTR by 2.96 tied-rank positions. RNG therefore gave the strongest joint QE /MTR result, while MST remained a close competitor on QE .

Table 1: Matched topology diagnostics for balanced quantization error and Mean Tied Rank. Positive paired effects favor the second topology in each contrast after applying metric directionality; lower raw values are better for both QE_B and MTR_B . Full confidence intervals, raw paired-test p-values, split-specific metrics, node utilization, and dead-node fraction are reported in Supplementary Table S12.

Profile	QE_B			MTR_B		
	MST vs hex	RNG vs hex	RNG vs MST	MST vs hex	RNG vs hex	RNG vs MST
Untuned	+0.269	+0.249	-0.020	-0.20	+2.44	+2.64
Tuned	+0.064	+0.052	-0.012	+22.71	+25.67	+2.96

5.4 Hyperparameter Tuning and Stability

5.4.1 Performance Gains from Hyperparameter Tuning

Having shown that the graph-topology effects persist when radius is varied under a common tuned non-radius configuration, we next quantify the broader effect of complete topology-specific tuning relative to the untuned reference.

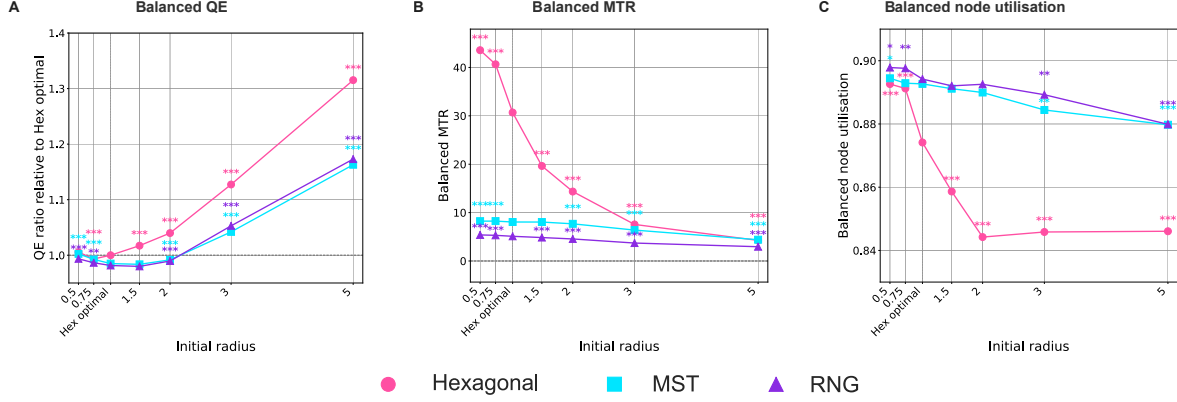
Figure 9: Initial-radius sensitivity

Figure 9: Initial-radius sensitivity under matched fixed configurations. A: dataset-balanced geometric-mean QE_B ratio relative to the matched hexagonal configuration at its selected radius ($r = 1.027$); lower values are better and the dashed horizontal line marks a ratio of 1. B: observed balanced Mean Tied Rank (MTR; lower is better). C: observed balanced node utilisation (higher is better). All panels summarize 14 datasets with 20 matched seeds per topology and radius. The dotted vertical line and **Hex optimal** tick mark $r = 1.027$. Significance markers use Benjamini-Hochberg adjusted tests comparing each non-anchor radius with $r = 1.027$ within topology across the 54 radius-topology-metric tests. Confidence intervals are omitted from the plotted panels for readability; Supplementary Table S15 reports the post hoc matched cross-topology QE_B estimates, intervals, and adjusted tests at every radius.

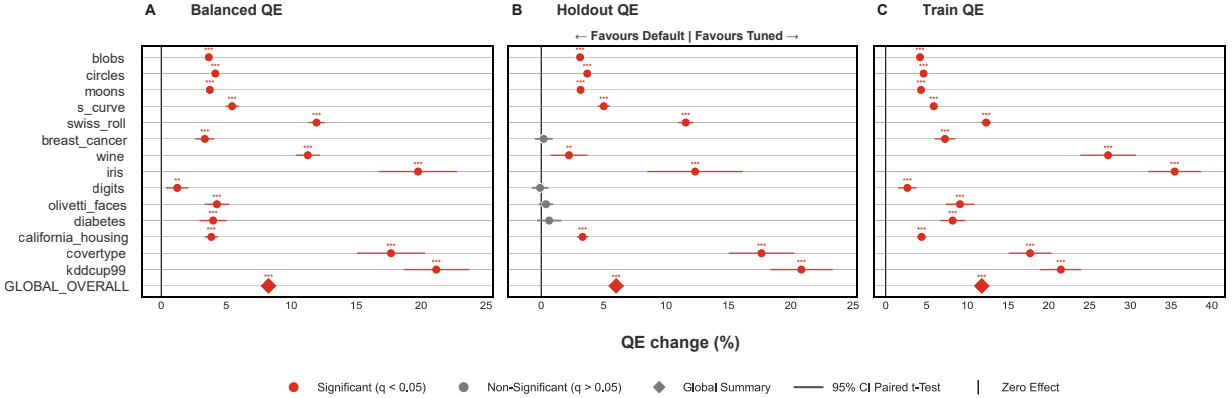
Figure 10: Tuned vs Default Across QE Metrics

Figure 10: Tuned configuration versus untuned reference QE comparison across QE_B , QE_H , and QE_T , pooled across all topology runs under the matched pairing keys. Positive values indicate the tuned configuration outperforms the untuned reference; the global overall row pools all matched tuned configuration/untuned reference pairs across datasets. Forest whiskers denote 95% paired t -test confidence intervals around the mean paired effect.

Fig. 10 compares the fixed tuned and untuned configurations using pooled topology-level QE summaries; topology-stratified results are provided in Supplementary Fig. S5.

The fixed tuned configurations had lower values for all three pooled QE metrics than the untuned references, but also had higher balanced MTR: by 24.34 tied-rank positions for hexagonal maps compared with 1.43 for MST and 1.11 for RNG. This profile-associated local-ordering trade-off was therefore much stronger for the fixed lattice and was not accompanied by lower node utilization.

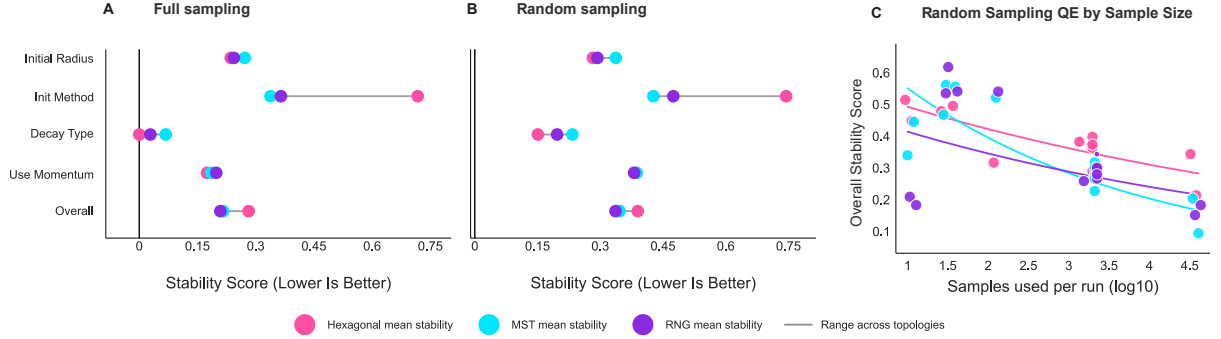
Figure 11: Hyperparameter Stability

Figure 11: Hyperparameter stability by sampling mode. A: selected parameter stability under full sampling for hexagonal, MST, and RNG topologies (lower stability score is better). B: selected parameter stability under random sampling for the same topologies. C: dataset-size stability regression under random sampling, using the selected parameter stability score against sample size (log10) across the included topology families.

The QE improvement was observed in every topology family.

5.4.2 Hyperparameter Stability Across Topology and Sampling

To assess whether the derived hyperparameters are robust across runs and datasets, we compare the within-topology seed-to-seed tuned-parameter drift (Section 4.3.2). Overall, MST and RNG exhibit less variability than hexagonal maps under both full and random sampling (Fig. 11A-B). Across these panels, full sampling is generally the more stable setting. Mirroring the sample-size-dependent QE performance, Fig. 11C shows that random-sampling hyperparameter stability also improves with increasing sample size.

5.5 Execution-Path Diagnostic

The quality analyses above used the in-memory pathway. Repeating the tuned diagnostics through Ray streaming with matched data and configurations produced small differences, none of which reached significance before or after Benjamini-Hochberg correction (Supplementary Table S14). The remaining differences may reflect floating-point accumulation order. The following speed benchmarks use the Ray pathway.

6 Speed Scaling

We next examine three aspects of FloatSOM runtime performance: the cost of full relative to random sampling, the scaling behavior obtained with additional GPUs, and the runtime implications of MST and RNG relative to a hexagonal topology.

6.1 Random versus Full Sampling Runtime

We report sample scaling runtime comparisons for full versus random sampling using the synthetic datasets as outlined in Section 4.2.1. We stratify by hexagonal, MST, and RNG, for 1, 2, and 4 GPUs (Fig. 12).

Across topologies, random sampling is faster than full sampling across all 1-, 2-, and 4-GPU comparisons, with similar proportional reductions at a given dataset size. With more GPUs, larger datasets can also be processed before timing out, although the runtime advantage of random over full narrows at the largest sample counts.

For the 1-GPU random sampling runs, the last successful 1-GPU run was at 100M samples, the same number of samples as run on full samples, despite random being much faster. We reason that the failed subsequent runs on larger samples were due to the transition from RAM operation to disk-backed operation. Even

Figure 12: Sampling Runtime

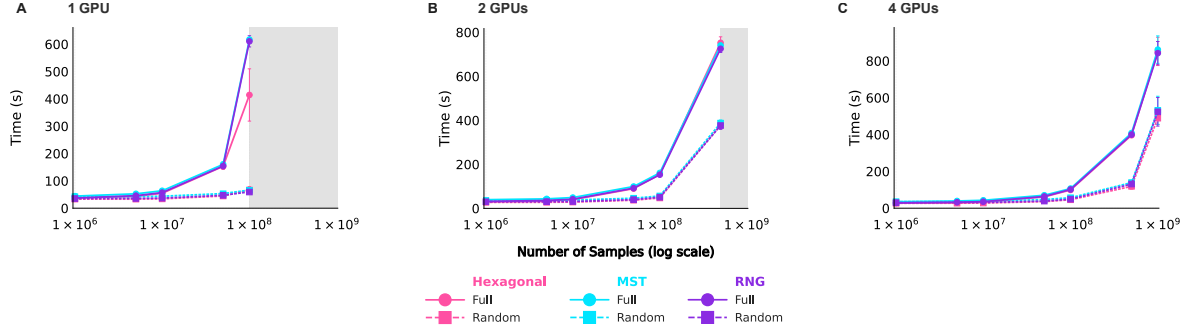


Figure 12: Sample scaling runtime comparison of full versus random sampling across $G \in \{1, 2, 4\}$ GPUs (A,B,C). Curves report mean wall-clock training time (s) under harmonized settings; error bars denote ± 1 standard deviation across $n = 3$ repeated runs per configuration. Color encodes topology (hexagonal, MST, RNG), and line style encodes sampling mode (full vs. random). Shaded x-axis regions indicate sample-size ranges that could not be run in that panel relative to the shared axis maximum due to timeouts. Lower values indicate faster execution.

in random sampling mode, the implementation still stages the full dataset initially and loads full chunks before the subsampler selects the training samples. Therefore, the relevant cost is no longer only the reduced number of selected samples. We therefore treat the 1-GPU random failure at 500M samples as a disk-mode systems limitation rather than as evidence against the general random versus full runtime ordering.

6.2 Multi-GPU topology scaling and Larger-Than-Memory context

To examine parallel scaling, we benchmarked FloatSOM under full sampling across $G \in \{1, 2, 4, 8\}$ GPUs using workload-scaling benchmarks that extend from standard in-memory settings to larger workloads that exceed available memory. Section 6.2 primarily concerns sample scaling. In that setting, the topologies exhibit similar scaling characteristics: as sample count increases, the GPU-count response and efficiency curves have similar qualitative shapes for RNG (Fig. 13B,E) and for the corresponding hexagonal and MST outputs (Fig. S6). Conversely, when the number of SOM nodes is increased (grid-size scaling), the topologies differ substantially. That grid-size regime is analyzed in Section 6.3, where MST and RNG take 8.19x and 27.07x the hexagonal runtime, respectively, at the largest tested grid size.

6.2.1 GPU Scaling and Larger-Than-Memory Runtime Performance

Fig. 13 shows that increasing GPU count improves performance in the sample scaling regime by increasing parallel throughput and delaying the transition to disk-backed execution. In the sample scaling benchmark, the 500,000,000 sample dataset requires disk backing under the 2-GPU configuration, whereas the 8-GPU configuration remains in RAM mode until the 1,000,000,000 sample dataset; when staging is still required, the disk-to-GPU path is distributed across more nodes. The 8-GPU RNG configuration processes 1,000,000,000 samples in 369.41 s (6.16 min). Note that this speed includes the time required to transfer data from shared storage to node-local shards, alongside the overhead associated with operating across multiple HPC nodes.

Grid-size scaling is the main exception: at the largest tested grid size (64), runtime decreases from 934.01 s (15.57 min) on 1 GPU to 880.83 s (14.68 min) on 8 GPUs, a 5.69% reduction. Once map-size and topology-refresh costs dominate, additional GPUs therefore contribute little further speedup.

6.2.2 GPU Scaling Efficiency

We next consider GPU efficiency under strong scaling, relative to ideal linear scaling. At smaller dataset sizes, efficiency is lower. As workload size increases, efficiency rises sharply and in some regions exceeds

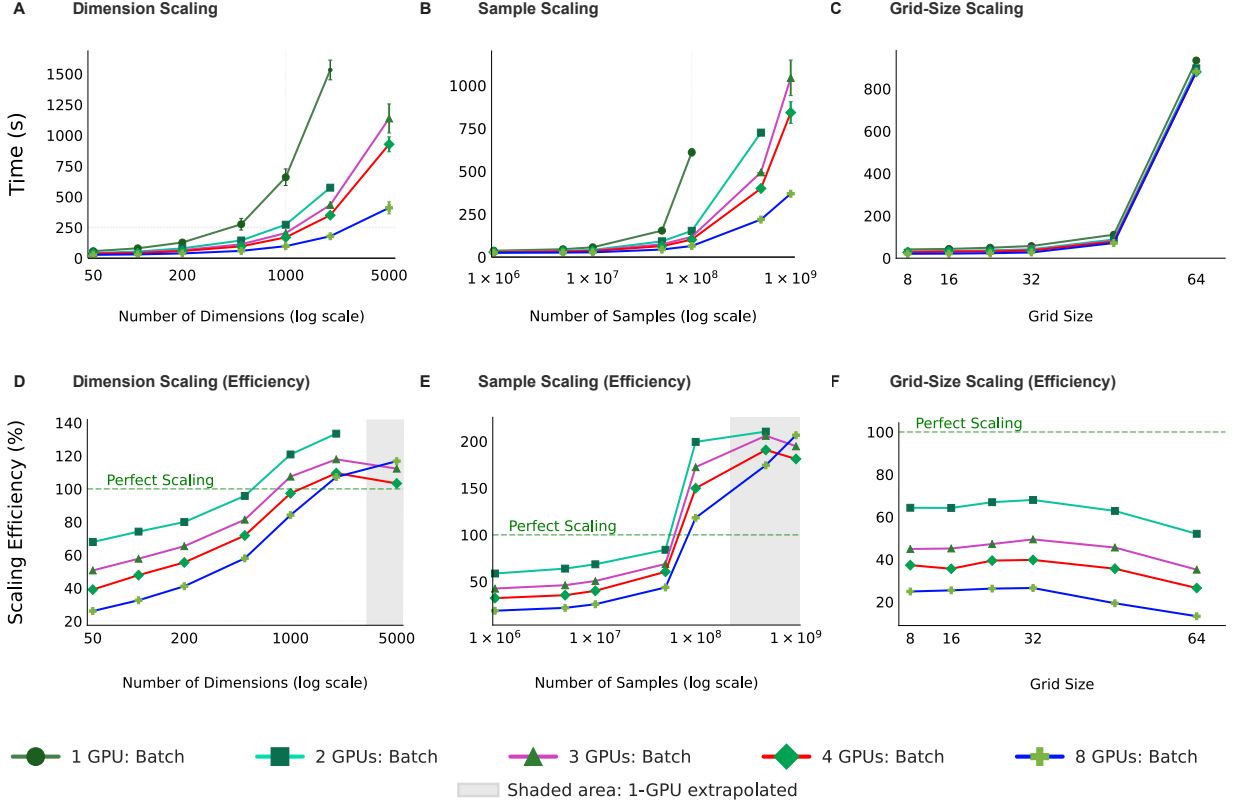
Figure 13: Multi-GPU Full-Batch Scaling (RNG)

Figure 13: Multi-GPU scaling across $G \in \{1, 2, 4, 8\}$ GPUs. Panels A-C show runtime (s) for dimension, sample, and grid size scaling workloads, respectively. Panels D-F show scaling efficiency for the same workloads, computed from the single-GPU baseline and the corresponding G -GPU runtime. Runtime error bars denote ± 1 standard deviation across $n = 3$ repeated runs per configuration; the 100% efficiency reference line indicates ideal linear scaling.

100%. When a direct 1-GPU baseline was unavailable at a given axis value, the efficiency denominator was constructed by local linear extrapolation from the last available 1-GPU point on that curve (Section 4.2), so some values should be interpreted with care if the underlying 1-GPU runtime is nonlinear over that range. Efficiencies above 100% should also be interpreted as a combined consequence of parallelism and a changed memory/data-staging regime, not as evidence of superlinear compute scaling. Together, Fig. 13 and Fig. S6 support the same qualitative sample-scaling efficiency trend across topologies. This does not extend to grid-size scaling; Section 6.3 shows that when node count increases, topology-dependent absolute runtimes diverge.

6.3 Topology Runtime Comparisons

With that systems context in place, we next compare topology runtimes across hexagonal, MST, and RNG configurations on 8 GPUs (Fig. 14).

In Fig. 14A-B, the topologies scale similarly as input complexity and data volume increase: even at the largest tested axis values, the maximum pairwise runtime spread remains modest at dimension scaling (4.70% at 5,000 dimensions) and sample scaling (3.26% at 1,000,000,000 samples).

However, when the grid itself is enlarged in Fig. 14C, topology-dependent runtime differences become readily evident. At the largest tested grid size (grid size 64), the 8-GPU mean runtimes are 32.54 s (0.54 min) for

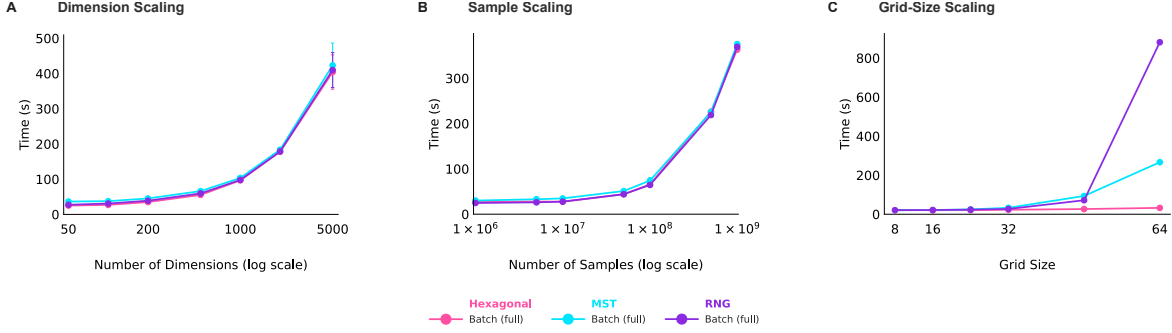
Figure 14: Topology Runtime Comparison (8 GPUs, Full Batch)

Figure 14: Topology runtime comparison at fixed $G = 8$ GPUs under full sampling. Panels A-C report mean wall-clock runtime (s) for dimension, sample, and grid size scaling workloads, respectively, with topology traces for hexagonal, MST, and RNG. Error bars denote ± 1 standard deviation across $n = 3$ repeated runs per configuration. The largest-axis 8-GPU topology runtime summaries are listed in Supplementary Table S11.

hexagonal, 266.45 s (4.44 min) for MST, and 880.83 s (14.68 min) for RNG, corresponding to 8-GPU MST and RNG runtimes that are 8.19x and 27.07x the hexagonal runtime, respectively.

7 Final FloatSOM RNG Comparison with XPySOM

Fig. 15 compares untuned hexagonal XPySOM with tuned FloatSOM RNG and therefore combines implementation, tuning, and topology effects (Mancini et al., 2020). Sections 5.1, 5.3, and 5.4 separate these components; Supplementary Figs. S7-S8 provide the corresponding tuned hexagonal and MST comparisons.

At the overall level, Fig. 15 shows median percentage improvements of QE_B (14.5%); QE_H (9.1%); and QE_T (22.5%) for tuned FloatSOM RNG relative to untuned hexagonal XPySOM, capturing the combined deployment effect of implementation, topology choice, and tuning on QE .

For the untuned hexagonal XPySOM reference in Fig. 15, workloads beyond the 10^8 -sample case were not processed because they exceeded available VRAM and XPySOM requires the full dataset to be loaded into memory. Taken together, Fig. 15 shows the point at which the workload size is large enough to warrant the extra startup overhead incurred by tuned FloatSOM RNG: tuned FloatSOM RNG delivers better QE than the untuned hexagonal XPySOM baseline, while also running faster and scaling to larger workloads.

8 Discussion

FloatSOM combines graph-based SOM topologies with distributed, out-of-memory GPU training. The benchmarks show how sampling, topology, tuning, and workload size affect both QE and runtime.

The trade-off between full and random sampling is scale dependent. Random sampling is less stable on small datasets, while above 10,000 samples we detect little paired QE difference. Full sampling is therefore preferable when stability is critical; random sampling offers higher throughput on larger datasets, although disk-backed operation limits its I/O advantage.

MST and RNG both improve QE over the fixed hexagonal lattice and are close competitors on QE , while RNG gives the stronger MTR result. This training-time use of MST must be distinguished from the more common post hoc use of an MST to connect or visualize prototypes learned by a conventional SOM, as in deployed workflows such as FlowSOM (Van Gassen et al., 2015). Earlier SOM literature considered MST-defined learning neighborhoods (Kangas et al., 1990), but FloatSOM recomputes MST or RNG connectivity

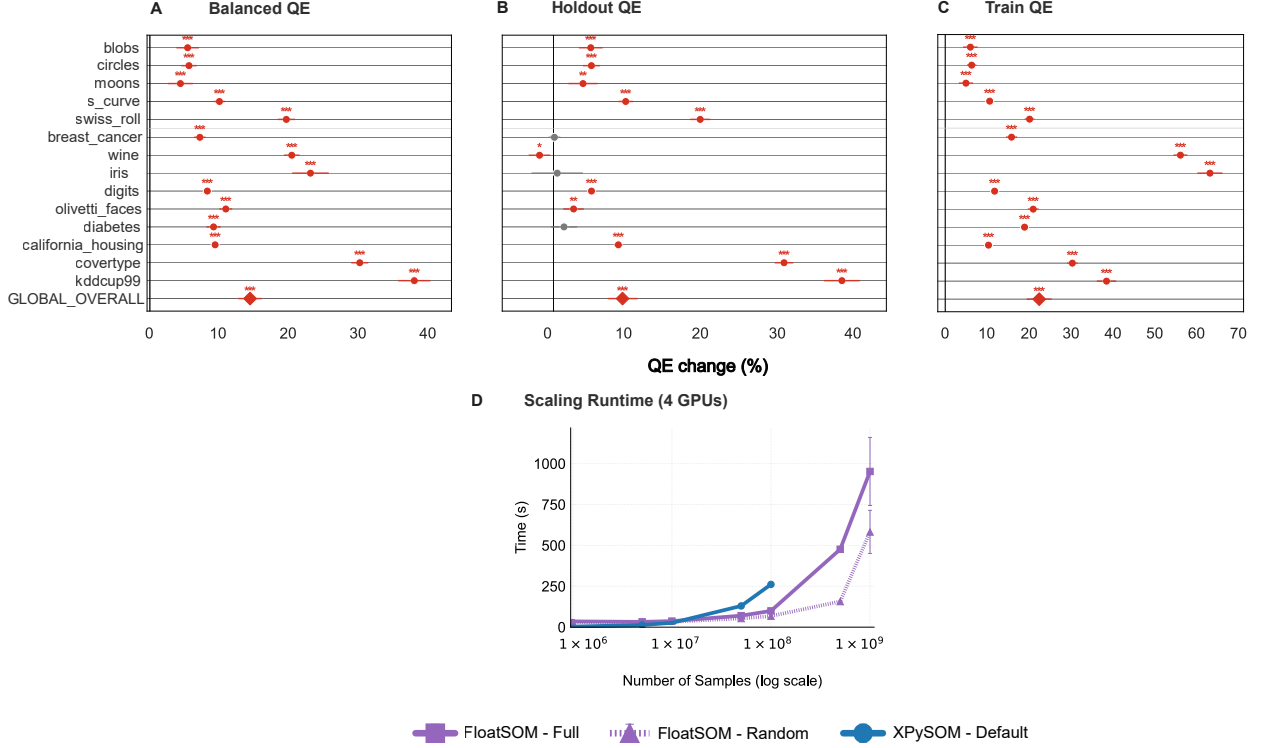
Figure 15: XPySOM (Default) v FloatSOM RNG

Figure 15: Integrated deployment comparison of untuned hexagonal XPySOM versus tuned FloatSOM RNG, combining implementation, tuning, and topology effects. Panels A-C compare QE_B , QE_H , and QE_T under full sampling; panel D provides the scaling/runtime context.

from the evolving node weights and uses graph hop distance directly in the neighborhood update throughout training. A post hoc graph can change only the displayed connectivity between already learned prototypes; it cannot change their locations or the resulting QE . The observed QE differentials therefore show that MST- and RNG-mediated training produces different, and on average more favourable, prototype locations in data space rather than merely a different set of connections drawn between the same nodes.

A geometric interpretation consistent with these results is that a fixed lattice imposes uniform, predetermined coupling: when a prototype moves, it can unnecessarily influence lattice neighbors that are unrelated in the learned data space, potentially displacing useful prototypes and increasing dead nodes. The apparently nonlocal hexagonal connections in the KDD Cup 99 projection (Fig. 5D) provide a visual illustration of this coupling and the associated restriction on independent prototype redistribution, subject to the distortions inherent in the two-dimensional PCA display. MST minimizes such coupling, allowing prototypes to redistribute more freely along irregular or elongated data structures. This freedom is also its limitation, because a tree cannot retain multiple locally appropriate connections where a concentrated region is better represented by a mesh. RNG occupies an intermediate position. Its nodes can influence several neighbors, but those connections arise from the evolving prototype geometry rather than being imposed before training; RNG can remain sparse where appropriate and form multiple connections in dense regions. The node-utilization and dead-node results in Fig. 9, together with QE and MTR, are consistent with this account, but they do not identify a unique causal mechanism (Kohonen, 2013; Kangas et al., 1990; Toussaint, 1980).

QE and MTR capture different properties: QE measures vector-quantization fidelity, while MTR measures local ordering between the first and second BMUs. The fixed QE -tuned profiles are associated with higher MTR in every topology, but the topological cost is far larger for hexagonal maps: balanced MTR is 24.34 tied-rank positions higher than in the untuned profile, compared with 1.43 for MST and 1.11 for RNG. The

matched radius sweep reinforces this distinction. Around $r = 1.027\text{--}1.5$, MST and RNG retain nearly flat QE while MTR is stable or improves and node utilization remains high; hexagonal maps instead show a pronounced exchange of lower MTR for worse QE and lower node utilization as radius increases (Fig. 9). Thus, the QE -tuned MST and RNG profiles are associated with nowhere near the ordering penalty observed for the fixed hexagonal lattice. Topology and hyperparameter choice should therefore be considered jointly rather than treating topology as a post-training visualization choice. The lower seed-to-seed variability of MST and RNG may likewise reflect their ability to rebuild connectivity from evolving node locations, unlike the fixed hexagonal lattice (Kohonen, 2013; Kangas et al., 1990).

The quality experiments use the in-memory pathway, whereas the scaling experiments use Ray. Within the distributed benchmark, scaling depends on workload size: overhead dominates small problems, while larger workloads benefit from parallel throughput and may remain in RAM when the 1-GPU case requires disk backing. Efficiencies above 100% reflect this change in memory regime rather than super-linear computation.

RNG is the preferred joint QE /MTR topology when topology overhead is acceptable; for QE alone, MST and RNG are close competitors and MST offers the lower-cost graph option. This recommendation depends on grid size. At grid size 64, hexagonal, MST, and RNG required 32.54, 266.45, and 880.83 s on 8 GPUs, respectively. Hexagonal is therefore the throughput-oriented choice for very large grids, while MST provides a compromise when a graph topology is desired. More generally, full sampling favors stability, random sampling favors throughput on large datasets, and additional GPUs are most useful when they keep the workload in RAM.

FloatSOM brings these topology and systems choices together in a single large-scale SOM framework.

Acknowledgments

This work was supported by computational resources provided by the Australian Government through the National Computational Infrastructure (NCI) under the ANU Merit Allocation Scheme.

We also acknowledge the computational services provided by the University of Bern, the University of Sydney, and the Walter and Eliza Hall Institute.

We thank Prof. Hanna Suominen for her input and advice.

References

- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A Next-generation Hyperparameter Optimization Framework, July 2019.
- Ayodeji A. Akinduko, Evgeny M. Mirkes, and Alexander N. Gorban. SOM: Stochastic initialization versus principal components. *Information Sciences*, 364–365:213–221, October 2016. ISSN 0020-0255. doi: 10.1016/j.ins.2015.10.013.
- Damminda Alahakoon, Saman Halgamuge, and Srinivasan Bala. Dynamic self-organizing maps with controlled growth for knowledge discovery. *IEEE Transactions on Neural Networks*, 11(3), 601–614. *Neural Networks, IEEE Transactions on*, 11:601–614, June 2000. doi: 10.1109/72.846732.
- Florent Forest, Mustapha Lebbah, Hanane Azzag, and Jérôme Lacaille. A Survey and Implementation of Performance Metrics for Self-Organized Maps. <https://arxiv.org/abs/2011.05847v1>, November 2020.
- Trung Ha, Joonas Nättilä, and Jordy Davelaar. aweSOM: A CPU/GPU-Accelerated Self-Organizing Map and Statistically Combined Ensemble Framework for Machine-Learning Clustering Analysis. *Journal of Open Source Software*, 10(108):7613, 2025. doi: 10.21105/joss.07613.
- Keiichi Ito, Ivo Couckuyt, Roberto d’Ippolito, and Tom Dhaene. Design space exploration using Self-Organizing Map based adaptive sampling. *Applied Soft Computing*, 43:337–346, June 2016. ISSN 1568-4946. doi: 10.1016/j.asoc.2016.02.036.
- Yoo-Jin Jang, Myung-Hoe Huh, and Mi-Ra Park. Use of Minimal Spanning Trees on Self-Organizing Maps. *Korean Journal of Applied Statistics*, 22, January 2009. doi: 10.5351/KJAS.2009.22.2.415.

- J. A. Kangas, T. K. Kohonen, and J. T. Laaksonen. Variants of self-organizing maps. *IEEE transactions on neural networks*, 1(1):93–99, 1990. ISSN 1045-9227. doi: 10.1109/72.80208.
- T. Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, September 1990. ISSN 1558-2256. doi: 10.1109/5.58325.
- Teuvo Kohonen. *Self-Organizing Maps*, volume 30 of *Springer Series in Information Sciences*. Springer, Berlin, Heidelberg, 2001. ISBN 978-3-540-67921-9 978-3-642-56927-2. doi: 10.1007/978-3-642-56927-2.
- Teuvo Kohonen. Essentials of the self-organizing map. *Neural Networks: The Official Journal of the International Neural Network Society*, 37:52–65, January 2013. ISSN 1879-2782. doi: 10.1016/j.neunet.2012.09.018.
- Miroslav Kratochvíl, Oliver Hunewald, Laurent Heirendt, Vasco Verissimo, Jiří Vondrášek, Venkata P Satagopam, Reinhard Schneider, Christophe Trefois, and Markus Ollert. GigaSOM.jl: High-performance clustering and visualization of huge cytometry datasets. *GigaScience*, 9(11):giaa127, November 2020. ISSN 2047-217X. doi: 10.1093/gigascience/giaa127.
- Joseph B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956. ISSN 0002-9939, 1088-6826. doi: 10.1090/S0002-9939-1956-0078686-7.
- Candace C. Liu, Noah F. Greenwald, Alex Kong, Erin F. McCaffrey, Ke Xuan Leow, Dunja Mrdjen, Bryan J. Cannon, Josef Lorenz Rumberger, Sricharan Reddy Varra, and Michael Angelo. Robust phenotyping of highly multiplexed tissue imaging data using pixel-level clustering. *Nature Communications*, 14(1):4618, August 2023. ISSN 2041-1723. doi: 10.1038/s41467-023-40068-5.
- Iván Machón-González and Hilario López-García. FLSOM with individual kernel radii formation and application to optimization of a pickling line. *Engineering Applications of Artificial Intelligence*, 23(3):411–419, 2010. doi: 10.1016/j.engappai.2010.01.016.
- Riccardo Mancini, Antonio Ritacco, Giacomo Lanciano, and Tommaso Cucinotta. XPySom: High-Performance Self-Organizing Maps. In *2020 IEEE 32nd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pp. 209–216, September 2020. doi: 10.1109/SBAC-PAD49847.2020.00037.
- Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation, OSDI’18*, pp. 561–577, USA, October 2018. USENIX Association. ISBN 978-1-931971-47-8.
- National Computational Infrastructure. HPC Systems | NCI. <https://nci.org.au/our-systems/hpc-systems>, n.d. Accessed 2026-03-22.
- Antonio Neme and Pedro Miramontes. Statistical Properties of Lattices Affect Topographic Error in Self-Organizing Maps. In *Artificial Neural Networks: Biological Inspirations – ICANN 2005*, pp. 427–432, Berlin, Heidelberg, 2005. Springer. doi: 10.1007/11550822_67.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. ISSN 1533-7928.
- Antonio Díaz Ramos, Ezequiel López-Rubio, and Esteban J. Palomo. The role of the lattice dimensionality in the self-organizing map. *Neural Network World*, 28(1):57–86, 2018. doi: 10.14311/NNW.2018.28.004.
- Gerasimos Spanakis and Gerhard Weiss. AMSOM: Adaptive Moving Self-organizing Map for Clustering and Visualization. <https://arxiv.org/abs/1605.06047v1>, May 2016.

- Godfried T. Toussaint. The relative neighbourhood graph of a finite planar set. *Pattern Recognition*, 12(4): 261–268, January 1980. ISSN 0031-3203. doi: 10.1016/0031-3203(80)90066-7.
- Sofie Van Gassen, Britt Callebaut, Mary J. Van Helden, Bart N. Lambrecht, Piet Demeester, Tom Dhaene, and Yvan Saeys. FlowSOM: Using self-organizing maps for visualization and interpretation of cytometry data. *Cytometry Part A*, 87(7):636–645, 2015. ISSN 1552-4930. doi: 10.1002/cyto.a.22625.
- Mahdi Vasighi and Homa Amini. A directed batch growing approach to enhance the topology preservation of self-organizing map. *Applied Soft Computing*, 55:424–435, June 2017. ISSN 1568-4946. doi: 10.1016/j.asoc.2017.02.015.
- Giuseppe Vettigli. JustGlowing/minisom, 2018.
- Leigh Wetmore, Malcolm I. Heywood, and A. Nur Zincir-Heywood. Speeding up the Self-Organizing Feature Map Using Dynamic Subset Selection. *Neural Processing Letters*, 22(1):17–32, August 2005. ISSN 1573-773X. doi: 10.1007/s11063-004-7775-6.
- Denis White and A. Ross Kiester. Topology matters: Network topology affects outcomes from community ecology neutral models. *Computers, Environment and Urban Systems*, 32(2):165–171, March 2008. ISSN 0198-9715. doi: 10.1016/j.compenvurbsys.2007.11.002.
- Peter Wittek, Shi Chao Gao, Ik Soo Lim, and Li Zhao. Somoclu: An Efficient Parallel Library for Self-Organizing Maps. *Journal of Statistical Software*, 78:1–21, June 2017. ISSN 1548-7660. doi: 10.18637/jss.v078.i09.

A Supplementary Methods and Tables (End Matter)

A.1 Supplementary Methods S1. Execution-path concordance protocol

The execution-path analysis used all 14 datasets, the 10 shared seeds 42–51, full sampling, and the hexagonal, MST, and RNG topologies on one NVIDIA V100 GPU. For each dataset–seed–topology unit, local CuPy and Ray streaming received the same standardized data, deterministic train–holdout split, topology-specific tuned parameters from Section 4.3.1, and evaluation metrics; execution profile was the only deliberately changed factor. Differences were defined as Ray streaming minus local CuPy.

For each topology, Supplementary Table S14 reports paired differences for balanced QE , MTR, node utilization, and dead-node fraction across 140 matched dataset–seed units. Mean differences and 95% confidence intervals were calculated from two-sided one-sample paired t -tests against zero, with Benjamini–Hochberg correction applied jointly across the 12 topology–metric tests.

A.2 Supplementary Tables

Supplementary Table S1: Dataset metadata and numbered point key for the Figure 4 sampling mode analysis.

dataset_index	dataset	dimension_count	sample_size	dataset_type
1	iris	4	150	real
2	wine	13	178	real
3	olivetti_faces	4096	400	real
4	diabetes	10	442	real
5	breast_cancer	30	569	real
6	digits	64	1797	real
7	california_housing	8	20640	real
8	blobs	2	30000	synthetic
9	circles	2	30000	synthetic
10	moons	2	30000	synthetic
11	s_curve	3	30000	synthetic
12	swiss_roll	3	30000	synthetic
13	kddcup99	41	494021	real
14	covertypes	54	581012	real

Supplementary Table S2: HDSSSOM pilot configuration summary for Figure 3.

field	value
pilot purpose	Initial HDSSSOM screening before the broader sampling comparison
configuration scope	Smaller hexagonal Optuna pilot
datasets	swiss_roll, moons, circles, blobs, s_curve, breast_cancer, wine, iris, digits, olivetti_faces
dataset count	10
seed count	5
sampling methods present	full, random, HDSSSOM
topology	hexagonal
algorithm family in campaign	full_batch
optimization split setup	evaluation-split=both, yielding QE_H and QE_T; Figure 3 reports paired QE_B
trials per scenario	200
main text figure slice	hexagonal / full vs HDSSSOM

Supplementary Table S3: Exact axis values used for the sample count, feature dimension, and grid size scaling benchmarks in Section 4.2.1.

scaling axis	exact values
sample count	10^6 , 5×10^6 , 10^7 , 5×10^7 , 10^8 , 5×10^8 , 10^9
feature dimension	50, 100, 200, 500, 1000, 2000, 5000
grid side length	8, 16, 24, 32, 48, 64

Supplementary Table S4. Untuned FloatSOM MST versus hexagonal XPySOM. The `dataset_index` column matches the numbered points in Supplementary Figure S1 panel D. Positive percentage values and positive signed effects favor FloatSOM; the compact embedded table lists wins as FloatSOM/XPySOM/ties. Complete row data are provided in `assets/tables/supp_table_s04.tsv`.

Supplementary Table S5. Untuned FloatSOM RNG versus hexagonal XPySOM. The `dataset_index` column matches the numbered points in Supplementary Figure S2 panel D. Positive percentage values and positive signed effects favor FloatSOM; the compact embedded table lists wins as FloatSOM/XPySOM/ties. Complete row data are provided in `assets/tables/supp_table_s05.tsv`.

Supplementary Table S6. Hexagonal FloatSOM–XPySOM implementation calibration. The `dataset_index` column matches the numbered points in Supplementary Figure S3 panel D. Positive percentage values and positive signed effects favor FloatSOM; the compact embedded table lists wins as FloatSOM/XPySOM/ties. Complete row data are provided in `assets/tables/supp_table_s06.tsv`.

Supplementary Table S7. Paired topology-comparison effects for hexagonal versus MST and hexagonal versus RNG across Balanced QE, Holdout QE, and Train QE. Rows list metric/dataset entries, including the OVERALL row. Effect columns report the paired mean percent improvement of hexagonal over the comparator topology, using hexagonal QE as the reference denominator; positive values favor hexagonal and negative values favor MST or RNG. The CI columns give the corresponding 95% paired *t*-test confidence intervals. Raw p-values are retained for audit, and q-values report Benjamini-Hochberg adjustment across the 42 dataset-level tests in each topology-comparison family (14 datasets by three QE endpoints, separately for MST and RNG); OVERALL rows are pooled summaries and retain q=NA. The embedded table is reproduced from `assets/tables/supp_table_topology_hex_vs_mst_rng_pvalues.tsv`. Complete row data are provided in `assets/tables/supp_table_s07.tsv`.

Supplementary Table S8. Figure 15 deployment comparison percent summary for tuned FloatSOM RNG versus untuned hexagonal XPySOM across QE_B , QE_H , and QE_T . Rows list per-dataset and GLOBAL_OVERALL entries with the plotted median percent change and 95% confidence interval. Complete row data are provided in `assets/tables/supp_table_s08.tsv`.

Supplementary Table S9. Supplementary Figure S7 deployment comparison percent summary for tuned FloatSOM hexagonal versus untuned hexagonal XPySOM across QE_B , QE_H , and QE_T . Rows list per-dataset and GLOBAL_OVERALL entries with the plotted median percent change and 95% confidence interval. Complete row data are provided in `assets/tables/supp_table_s09.tsv`.

Supplementary Table S10. Supplementary Figure S8 deployment comparison percent summary for tuned FloatSOM MST versus untuned hexagonal XPySOM across QE_B , QE_H , and QE_T . Rows list per-dataset and GLOBAL_OVERALL entries with the plotted median percent change and 95% confidence interval. Complete row data are provided in `assets/tables/supp_table_s10.tsv`.

Supplementary Table S11. Figure 14 topology runtime summary at the largest common 8-GPU axis value for the dimension, sample, and grid size scaling workloads. Rows report the plotted 8-GPU mean runtimes for hexagonal, MST, and RNG, together with the fastest and slowest topology at that axis value and the maximum pairwise runtime spread. Complete row data are provided in `assets/tables/supp_table_s11.tsv`.

Supplementary Table S12. Matched topology diagnostic paired summaries across tuned and untuned profiles. Rows report pooled paired effects from the final matched random-seed topology diagnostic benchmark. **Effect favoring comparator** is oriented so positive values favor the comparator after applying each metric’s directionality; lower is better for QE, MTR, and dead-node fraction, while higher is better for node utilization. Because these are pre-specified pooled paired summaries rather than dataset-level test families, the table reports raw paired-test p-values. **comp/ref/tie** gives the number of comparator-favoring, reference-favoring, and tied matched pairs. Metric suffixes denote holdout (**_H**), train (**_T**), and balanced train-holdout (**_B**) summaries. The tuned profile uses the deployable tuned configurations reported in the main text. Complete row data are provided in `assets/tables/supp_table_s12.tsv`.

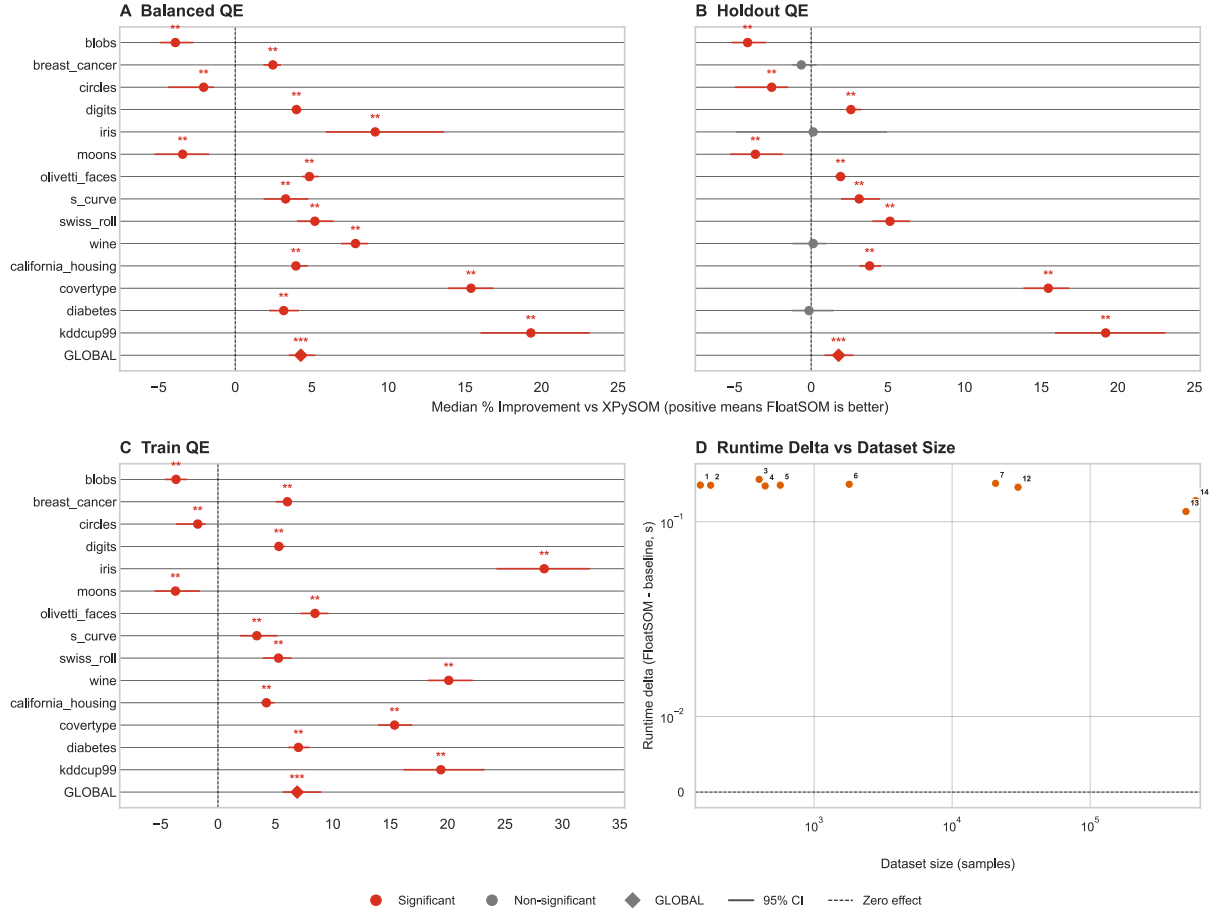
Supplementary Table S13. Full matched topology diagnostic means by profile, dataset, and topology. Rows report the mean value across the final matched random seeds for each profile, dataset, topology, and full-sampling setting. **untuned** denotes the untuned reference profile using XPySOM-like default hyperparameters and **tuned** denotes the fixed QE-tuned profile. Lower is better for QE, MTR, and dead-node fraction; higher is better for node utilization. Metric suffixes denote holdout (**_H**), train (**_T**), and balanced train-holdout (**_B**) summaries. Complete row data are provided in `assets/tables/supp_table_s13.tsv`.

Supplementary Table S14. Matched local CuPy and Ray streaming execution-path diagnostics. Values are paired differences computed as Ray streaming minus local CuPy for matched dataset, seed, topology, and tuned configuration. Each row summarizes the paired differences across 14 datasets and 10 seeds. The table reports the mean paired difference, standard deviation of the paired difference, 95% confidence interval, and Benjamini-Hochberg adjusted q-value. Complete row data are provided in `assets/tables/supp_table_s14.tsv`.

Supplementary Table S15. Post hoc matched cross-topology balanced-QE contrasts across the initial-radius sweep. For each topology pair and radius, seed-level log QE_B ratios were averaged within each dataset and then tested across the 14 equally weighted dataset effects. **QE ratio** is the exponentiated mean log ratio (comparator/reference), so values below 1 and positive improvement values favor the comparator. Confidence intervals are exponentiated 95% paired t intervals. Benjamini-Hochberg q-values adjust across all 21 topology-pair-radius tests. Complete row data are provided in `assets/tables/supp_table_s15.tsv`.

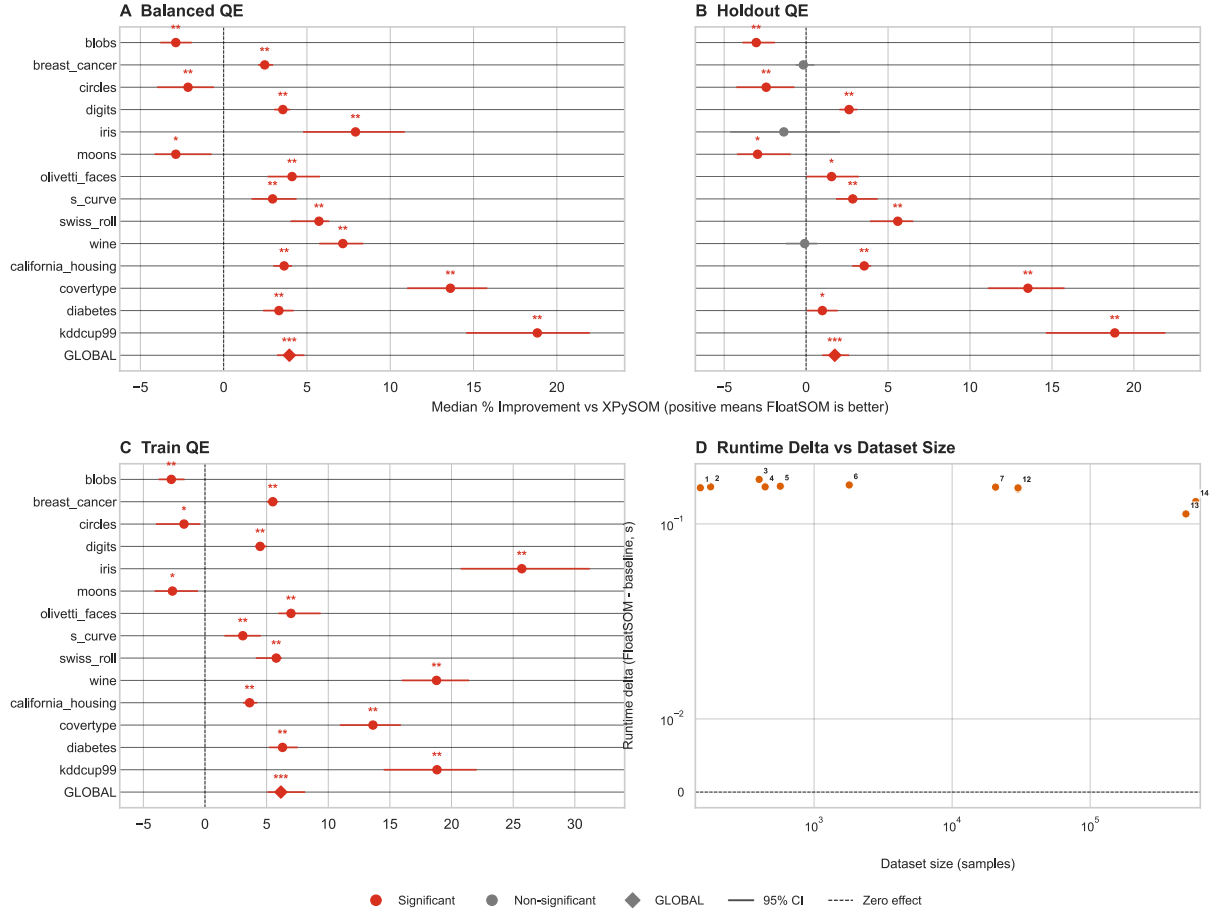
B Supplementary Figures (End Matter)

Supplementary Figure S1: FloatSOM vs XPySOM Calibration (Default Settings, MST)



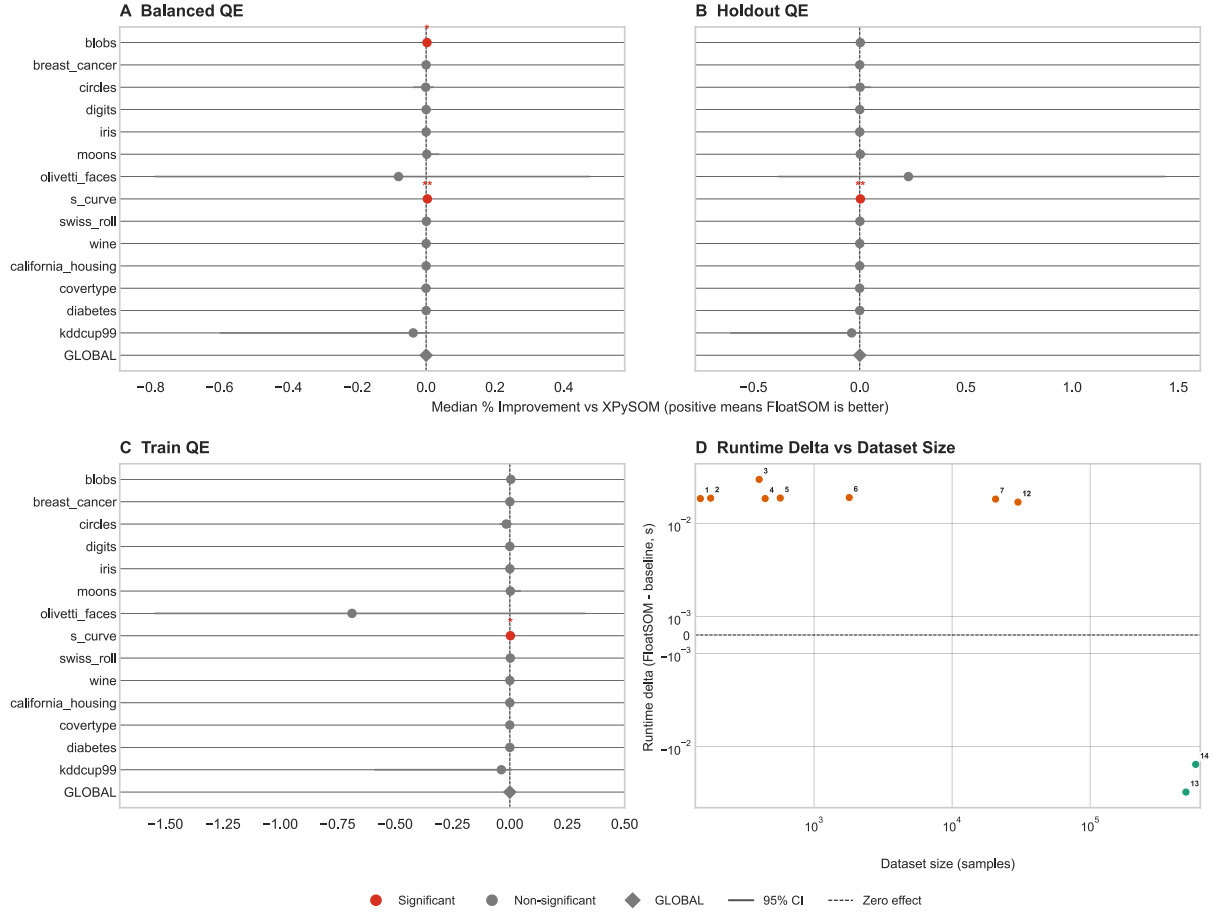
Supplementary Figure S1: Untuned FloatSOM MST versus hexagonal XPySOM. Panels A-C report paired QE effects for QE_B , QE_H , and QE_T . Panel D reports dataset-level median runtime deltas against dataset size, where each numbered dot is the median matched-seed value of **FloatSOM time** - **XPySOM time**; negative values favor FloatSOM and positive values favor XPySOM. The point numbers map to Supplementary Table S4. Forest whiskers denote exact distribution-free 95% intervals for the paired Wilcoxon location estimate.

Supplementary Figure S2: FloatSOM vs XPySOM Calibration (Default Settings, RNG)



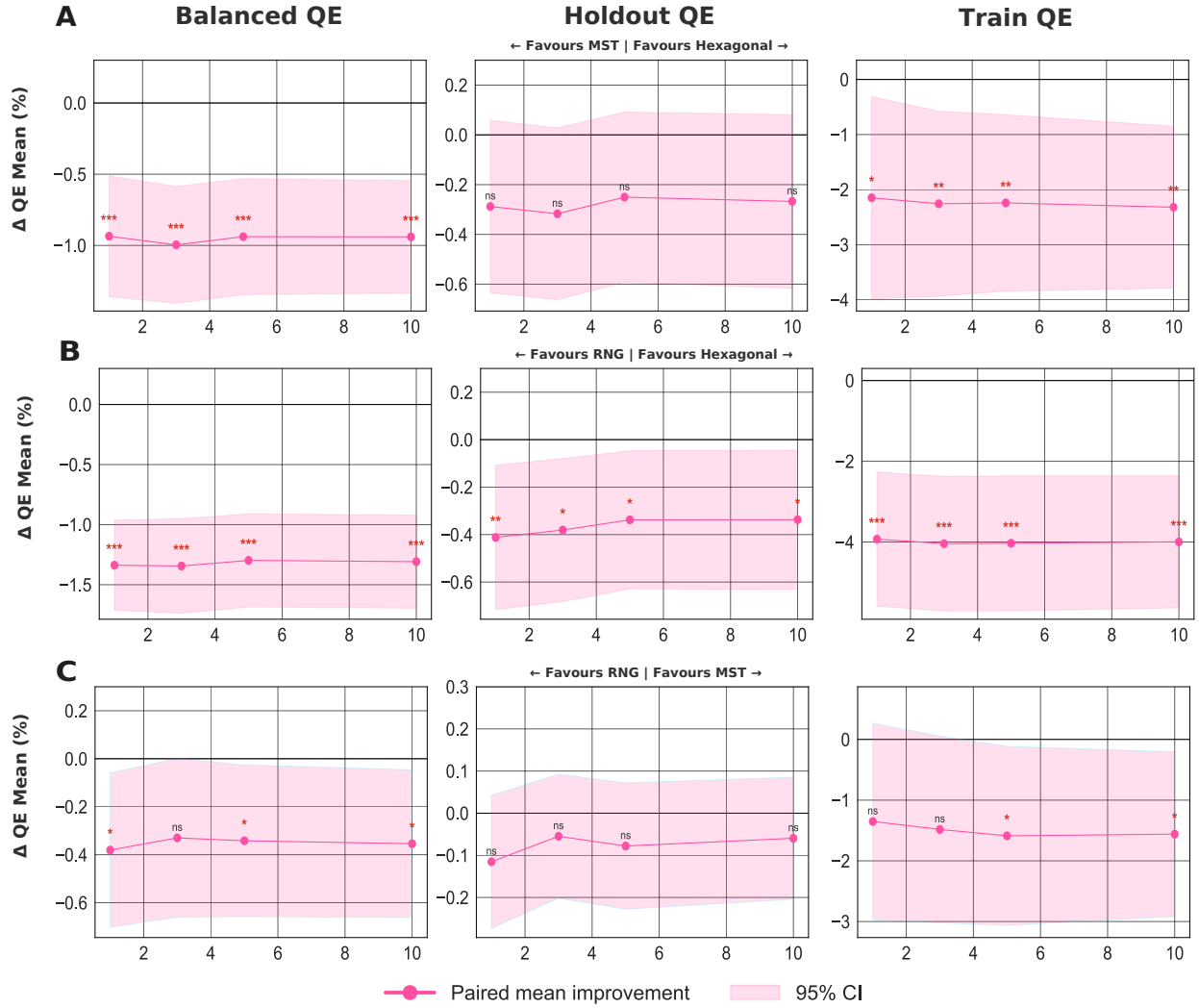
Supplementary Figure S2: Untuned FloatSOM RNG versus hexagonal XPySOM. Panels A-C report paired QE effects for QE_B , QE_H , and QE_T . Panel D reports dataset-level median runtime deltas against dataset size, where each numbered dot is the median matched-seed value of **FloatSOM time** - **XPySOM time**; negative values favor FloatSOM and positive values favor XPySOM. The point numbers map to Supplementary Table S5. Forest whiskers denote exact distribution-free 95% intervals for the paired Wilcoxon location estimate.

Supplementary Figure S3: FloatSOM vs XPySOM Calibration (Default Settings, Hexagonal)



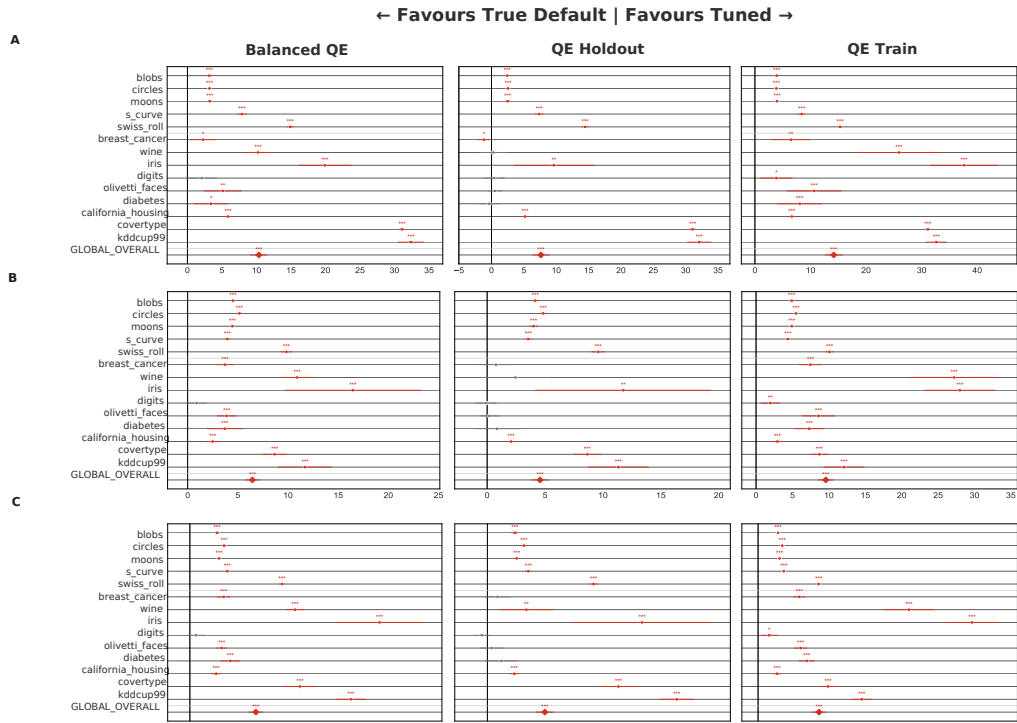
Supplementary Figure S3: Hexagonal FloatSOM–XPySOM implementation calibration. Panels A–C report paired QE effects for QE_B , QE_H , and QE_T . Panel D reports dataset-level median runtime deltas against dataset size, where each numbered dot is the median matched-seed value of `FloatSOM time` – `XPySOM time`; negative values favor FloatSOM and positive values favor XPySOM. The point numbers map to Supplementary Table S6. Forest whiskers denote exact distribution-free 95% intervals for the paired Wilcoxon location estimate.

Supplementary Figure S4: Topology Sensitivity by Contrast

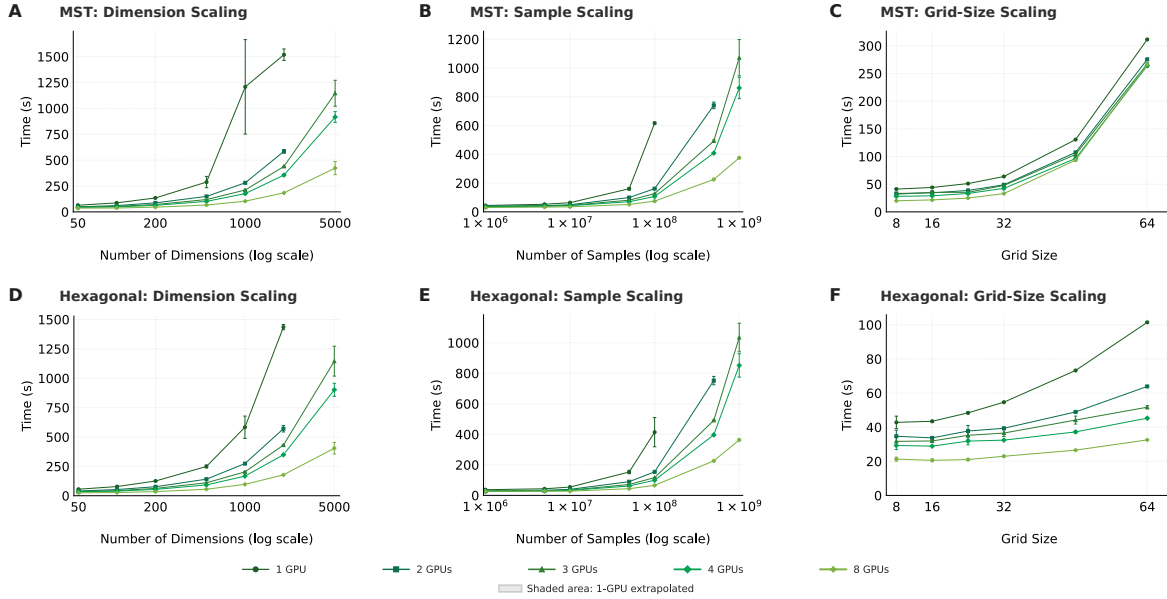


Supplementary Figure S4: Topology sensitivity analyses under full sampling. A: hexagonal versus MST; B: hexagonal versus RNG; C: MST versus RNG. Within each row, columns report matched top- k paired sensitivity analyses for QE_B , QE_H , and QE_T at $k \in \{1, 3, 5, 10\}$; directional labels indicate which topology is favored.

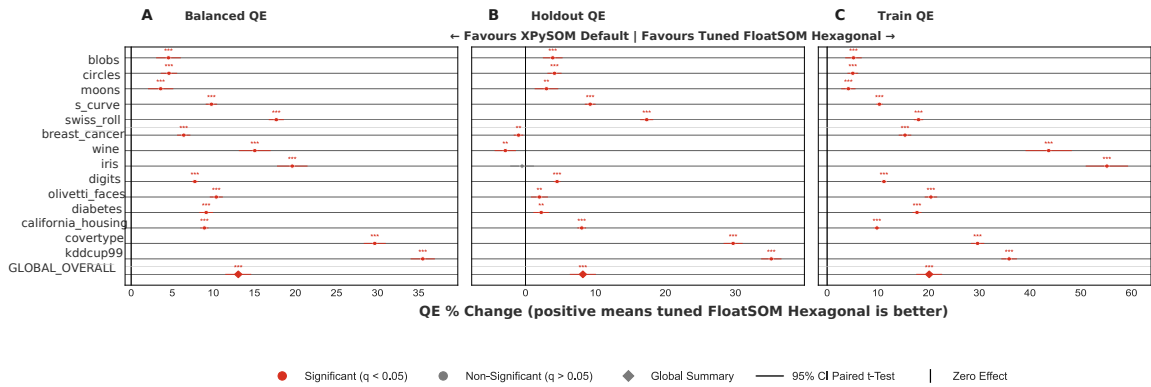
Supplementary Figure S5: Tuned vs Untuned by Topology



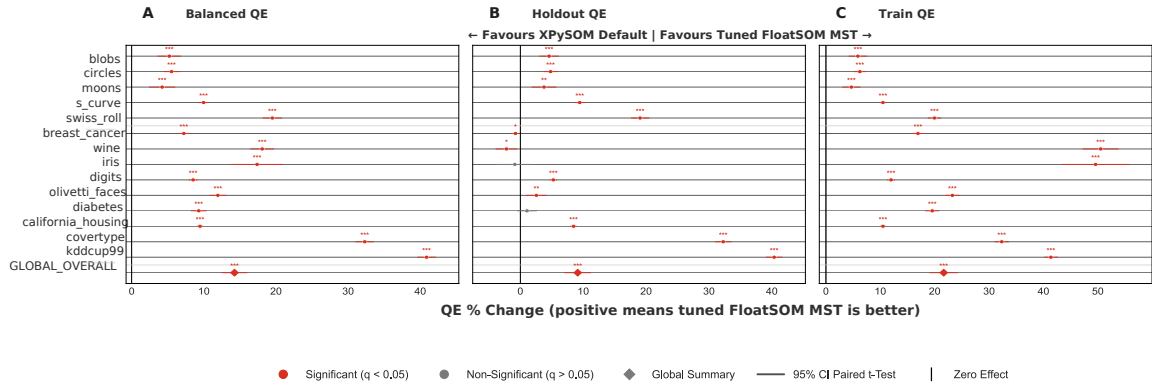
Supplementary Figure S5: Tuned versus untuned fixed-configuration comparisons by topology. A: hexagonal; B: MST; C: RNG. Within each row, columns report QE_B , QE_H , and QE_T for 14 datasets and 20 seeds under matched dataset–seed–topology–split keys. Positive values indicate that the tuned configuration outperforms the untuned reference; profile definitions are given in Section 4.3.1.

Supplementary Figure S6: GPU Scaling by Workload (A-C: MST; D-F: Hexagonal)

Supplementary Figure S6: Full GPU-count scaling context for MST and hexagonal under matched full sampling settings. A-C: MST dimension, sample, and grid-size scaling; D-F: hexagonal dimension, sample, and grid-size scaling. Curves correspond to $G \in \{1, 2, 4, 8\}$ GPUs and report mean wall-clock runtime (s) with ± 1 standard-deviation error bars across $n = 3$ repeated runs per configuration.

Supplementary Figure S7: Tuned Hexagonal vs XPySOM Default

Supplementary Figure S7: Deployment comparison of untuned hexagonal XPySOM versus tuned FloatSOM hexagonal. A: QE_B ; B: QE_H ; C: QE_T under the matched dataset/seed comparison keys. Positive values indicate tuned FloatSOM hexagonal outperforms untuned hexagonal XPySOM. Forest whiskers denote 95% paired t -test confidence intervals around the mean paired effect. Per-dataset and GLOBAL_OVERALL panel summaries are listed in Supplementary Table S9.

Supplementary Figure S8: Tuned MST vs XPySOM Default

Supplementary Figure S8: Deployment comparison of untuned hexagonal XPySOM versus tuned FloatSOM MST. A: QE_B ; B: QE_H ; C: QE_T under the matched dataset/seed comparison keys. Positive values indicate tuned FloatSOM MST outperforms untuned hexagonal XPySOM. Forest whiskers denote 95% paired t -test confidence intervals around the mean paired effect. Per-dataset and GLOBAL_OVERALL panel summaries are listed in Supplementary Table S10.